MATERIAL DE ESTUDO

# VIBE CODING

## Guia de Estudo — Edição Ilustrada



Do “esqueça que o código existe” ao desenvolvimento com agentes de IA: termos, técnicas, ferramentas, riscos e boas práticas — explicados camada por camada.

13 MÓDULOS · 100+ TERMOS · PROMPTS PRONTOS · SEGURANÇA & RISCOS

EDIÇÃO DE ESTUDO · VERSÃO OUT/2026

ÍNDICE

# SUMÁRIO

Um mapa completo do que sustenta o desenvolvimento com IA — da origem do termo "vibe coding" às técnicas de prompt, contexto, agentes, testes, segurança e boas práticas, passando pelas ferramentas mais usadas.


— **Como usar este guia** — 03
Legenda dos quadros, trilhas de estudo e pré-requisitos

01 **O que é Vibe Coding** — 05
Origem do termo, definições (estrita e ampla), espectro de autonomia e quando usar

02 **Como a IA escreve código** — 08
LLMs, tokens, contexto, raciocínio, ferramentas, agentes e os limites do modelo

03 **Fluxo de trabalho** — 11
O ciclo descrever → gerar → rodar → ajustar, Git como rede de segurança e como sair de loops

04 **Prompts para código** — 14
Anatomia de um bom pedido, técnicas, modelos prontos e erros comuns

05 **Engenharia de contexto** — 17
Janela de contexto, context rot, arquivos de regras (AGENTS.md), memória e compactação

06 **Estendendo agentes** — 20
MCP, skills, hooks, subagentes, plugins, permissões e trabalho em paralelo

07 **Planejar antes de codar** — 23
Plan mode, especificações (spec-driven development), tarefas pequenas e TDD

08 **Ferramentas do ecossistema** — 26
Construtores de apps, editores com IA, agentes de terminal e nuvem, modelos e como escolher

09 **Testes, depuração e revisão** — 29
Pirâmide de testes, TDD com IA, revisão de diffs, depuração e sinais de código gerado ruim

10 **Segurança, riscos e ética** — 32
Vulnerabilidades comuns, slopsquatting, prompt injection, OWASP, LGPD e uso responsável

11 **O que a pesquisa diz** — 36
Estudos sobre produtividade, qualidade, confiança e aprendizado com assistentes de código

12 **Boas práticas e armadilhas** — 39
Checklists, antipadrões, custos, aprendizado e quando assumir o volante

13 **Mão na massa** — 42
Projeto-guia em 8 etapas com prompts prontos, trilha de 4 semanas, templates e revisão

— **Glossário rápido** — 45
Mais de 100 termos de A a Z para consulta rápida

— **Fontes e leituras** — 49
Referências usadas na pesquisa e materiais para se aprofundar

ANTES DE COMEÇAR

# — COMO USAR ESTE GUIA

Um mapa rápido para tirar o máximo do material — do primeiro contato à prática segura.

Este guia reúne os principais **termos, técnicas, conceitos e ferramentas** do vibe coding — do que o termo significa até um projeto prático. A ordem vai do "o que é" ao "como fazer com segurança", mas cada módulo funciona sozinho.

## MAPA DO GUIA

**1 · ENTENDER** — Fundamentos
- 01 O que é vibe coding
- 02 Como a IA escreve código

**2 · FAZER** — Prática
- 03 Fluxo de trabalho
- 04 Prompts para código
- 05 Engenharia de contexto

**3 · ESCALAR** — Agentes e processo
- 06 Estendendo agentes
- 07 Planejar antes de codar

**4 · ESCOLHER** — Ferramentas
- 08 Ferramentas do ecossistema

**5 · PROTEGER** — Qualidade, segurança e evidências
- 09 Testes, depuração e revisão
- 10 Segurança, riscos e ética
- 11 O que a pesquisa diz

**6 · PRATICAR** — Consolidação
- 12 Boas práticas e armadilhas
- 13 Mão na massa

**Seis blocos, do entendimento à prática.** Os números dentro de cada bloco são os módulos; o sumário mostra a página de cada um.

## LEGENDA DOS QUADROS

**Em resumo:** a ideia central de uma seção, em uma ou duas frases. Ótimo para revisar.

**Boa prática:** um hábito que funciona na prática e vale virar rotina.

**Atenção:** uma armadilha comum ou um risco real. Leia antes de seguir em frente.



**DICA DA CORUJA**

o conselho de quem já errou nisso: um atalho, um critério de decisão ou uma regra de bolso.

**EXEMPLO DE PROMPT**

Blocos escuros como este são **modelos para copiar**. Troque o que está entre [colchetes] pelo seu caso e ajuste ao seu projeto.

## — COMO USAR ESTE GUIA

CONTINUAÇÃO

### TRILHAS DE ESTUDO

| SEU PERFIL | COMECE POR | DEPOIS | FIXE COM |
|---|---|---|---|
| **Iniciante: nunca programei** | **p. 5** → **p. 11** → **p. 14** | **p. 26** → **p. 39** | Projeto-guia (**p. 42**) e glossário |
| **Já programo e uso IA às vezes** | **p. 8** → **p. 17** → **p. 20** | **p. 23** → **p. 29** | Perguntas de revisão (**p. 42**) |
| **Profissional: quero processo e controle de risco** | **p. 17** → **p. 23** → **p. 29** | **p. 32** → **p. 36** → **p. 39** | Checklists de segurança e entrega |
| **Revisão rápida em 1 hora** | Decálogo (**p. 39**) | Quadros “Em resumo” de cada módulo | Glossário (**p. 45**) |

### COMO LER O TEXTO

**Termos e código**

*Termos em inglês* aparecem em itálico na primeira vez. `Comandos`, arquivos e trechos de código ficam em fonte monoespaçada.

**Referências de página**

Indicações como **p. 32** levam ao início do módulo citado. No PDF digital, elas, o sumário e as fontes são clicáveis.

**Datas e números**

As ferramentas citadas são um retrato de **2026**. Preços, versões e estudos mudam rápido: confirme nas fontes (**p. 49**).

### O QUE VOCÊ PRECISA SABER ANTES

**Não é pré-requisito**

- Saber programar em uma linguagem específica
- Conhecer o nome das ferramentas citadas
- Ter lido sobre IA ou machine learning

**Ajuda bastante**

- Ideia do que são arquivo, pasta e terminal
- Noções de Git: commit, branch e pull request (revisão na **p. 11**)
- Noções de IA generativa: modelo, token, prompt (revisão na **p. 8**)

**DICA DA CORUJA**

Estude com o editor aberto. Cada técnica fica mais clara quando você a **testa em um projeto pequeno** — e o módulo 13 sugere exatamente um.

MÓDULO 01 · FUNDAMENTOS

# 01 O QUE É VIBE CODING

De onde veio o termo, o que ele significa de verdade e onde ele se encaixa no desenvolvimento de software.

**Vibe coding** é programar descrevendo, em linguagem natural, o que você quer — e deixar um modelo de IA escrever o código. Na definição original, você avalia o resultado **rodando o programa**, não lendo cada linha.

## ORIGEM DO TERMO

2 DE FEVEREIRO DE 2025

### O post de Karpathy

Andrej Karpathy (cofundador da OpenAI e ex-diretor de IA da Tesla) descreveu uma "nova forma de programar": se entregar totalmente às *vibes*, esquecer que o código existe, aceitar tudo sem ler os diffs e **colar as mensagens de erro de volta** até funcionar. Ele mesmo situou a técnica em "projetos descartáveis de fim de semana".

DICIONÁRIOS

### Por que pegou

O Merriam-Webster registra o termo entre gírias e tendências ("escrever código de forma um tanto descuidada, com ajuda de IA") e o Collins o elegeu **palavra do ano de 2025**. Virou rótulo para qualquer programação com IA — o que gerou debate sobre o significado.

**Em resumo:** vibe coding = **intenção em linguagem natural → código gerado → teste rodando → ajuste por conversa**. O foco sai da sintaxe e vai para o *resultado*.

## DEFINIÇÃO ESTRITA × DEFINIÇÃO AMPLA

| VISÃO | O QUE SIGNIFICA | EXEMPLO |
|---|---|---|
| **Estrita (original)** | Aceitar o código sem revisar nem entender por completo; julgar só pelo comportamento do app. | Protótipo de fim de semana, script descartável. |
| **Ampla (popular)** | Qualquer desenvolvimento em que uma IA escreve a maior parte do código a partir de prompts. | Uso diário de agentes em projetos reais. |
| **Engenharia com IA** | A IA escreve, mas você especifica, testa, revisa e entende — como num time com pessoas. | Fluxo com plano, testes e revisão de PR. |

REGRA DE OURO DE SIMON WILLISON

Se você revisou, testou e consegue explicar o que o código faz, **isso não é vibe coding** — é desenvolvimento de software com a IA como assistente. Willison só faz commit de código que conseguiria explicar a outra pessoa.

## 01 O QUE É VIBE CODING

CONTINUAÇÃO




**Quadrantes:** a mesma ferramenta serve a usos muito diferentes. O que muda é *quanto você verifica* e *quanta autonomia* dá à IA.

### SOFTWARE 1.0, 2.0 E 3.0 (KARPATHY)

CÓDIGO ESCRITO POR PESSOAS

**Software 1.0**

Instruções explícitas em Python, C++, JavaScript... Você escreve a lógica; o computador executa exatamente o que foi escrito.

PESOS APRENDIDOS

**Software 2.0**

O "programa" são os pesos de uma rede neural, aprendidos a partir de dados (visão computacional, tradução, jogos).

PROMPTS COMO PROGRAMA

**Software 3.0**

Modelos de linguagem (LLMs) se tornam programáveis em **linguagem natural**. "O inglês é a nova linguagem de programação" — ideia que Karpathy resumiu já em 2023.

### A ESCADA DE AUTONOMIA




**Cada degrau** entrega mais autonomia à IA — e exige mais verificação de você. O vibe coding costuma acontecer nos degraus 4 e 5.

# 01 O QUE É VIBE CODING

CONTINUAÇÃO

## QUEM USA — E PARA QUÊ

SEM FORMAÇÃO EM PROGRAMAÇÃO

### Iniciantes

Criam protótipos, ferramentas pessoais e automações simples. O ganho é **começar**; o risco é não perceber o que está errado.

COM EXPERIÊNCIA

### Pessoas desenvolvedoras

Usam para scripts, MVPs, *boilerplate*, testes e exploração de ideias — e revisam o que importa.

VELOCIDADE DE VALIDAÇÃO

### Startups e times

Segundo Jared Friedman (YC), cerca de 25% das startups do lote W25 tinham ~95% do código gerado por IA (sem contar *boilerplate*).

## QUANDO FAZ SENTIDO — E QUANDO NÃO

### Bom para

- Protótipos, demos, hackathons e ideias para testar rápido
- Scripts de uso único e automações pessoais
- Ferramentas internas pequenas e de baixo risco
- Aprender: ver o código funcionando e perguntar "por quê?"
- Telas, layouts e *boilerplate* repetitivo

### Evite (ou exija engenharia completa)

- Pagamentos, saúde, jurídico e dados pessoais sensíveis
- Autenticação, permissões e criptografia
- Código que você vai manter por anos ou que outras pessoas vão herdar
- Sistemas sem testes, sem revisão e sem backup
- Qualquer coisa que você não saberia explicar a um colega

## VIBE CODING × ENGENHARIA AGÊNTICA

| ASPECTO | VIBE CODING (ESTRITO) | ENGENHARIA AGÊNTICA |
|---|---|---|
| **Objetivo** | Chegar rápido a algo que funciona | Entregar software confiável e mantível |
| **Revisão do código** | Mínima ou nenhuma | Revisão de diffs, testes e leitura crítica |
| **Papel da pessoa** | Pede, testa, cola erros | Especifica, orquestra agentes e supervisiona |
| **Planejamento** | Improviso por conversa | Plano, especificação e tarefas pequenas |
| **Responsabilidade** | Difusa ("a IA fez") | Sua — como se você tivesse escrito |
| **Termo associado** | "Vibe coding" (Karpathy, fev/2025) | "Vibe engineering" (Willison, out/2025) · "Agentic engineering" (Karpathy, fev/2026) |

## LINHA DO TEMPO

**2 FEV 2025**
Post que cunha o termo "vibe coding".

**JUN 2025**
Karpathy apresenta "Software 3.0" na YC AI Startup School.

**JUL 2025**
Agente da Replit apaga um banco de produção durante um "code freeze".

**7 OUT 2025**
Simon Willison propõe "vibe engineering".

**6 NOV 2025**
Collins elege "vibe coding" a palavra do ano.

**FEV 2026**
Karpathy sugere "agentic engineering" para o trabalho profissional com agentes.

MÓDULO 02 · FUNDAMENTOS

# 02 COMO A IA ESCREVE CÓDIGO

O que acontece por baixo do capô — e por que entender isso muda a forma de pedir e de revisar.

Um **LLM** (*Large Language Model*, modelo de linguagem de grande porte) não "entende" código como uma pessoa: ele **prevê o próximo trecho mais provável** a partir de tudo o que está em seu contexto. Entender isso explica tanto o poder quanto os erros dessas ferramentas.

## DA PREVISÃO DE TEXTO AO CÓDIGO

COMO O MODELO "LÊ"

### Tokens e previsão

O texto (e o código) é dividido em **tokens** — pedaços de palavras, símbolos e espaços. A cada passo o modelo calcula probabilidades para o próximo token, escolhe um e repete. Para ele, Python, SQL e português são só "idiomas" com padrões diferentes.

DE ONDE VEM A HABILIDADE

### Treinamento

**Pré-treino** em enormes volumes de texto e código; depois, **pós-treino** com instruções, feedback humano (RLHF) e aprendizado por reforço com recompensas verificáveis — como *testes que passam*. Código é um terreno fértil: é formal, abundante e dá para checar se funcionou.

**Em resumo:** o modelo gera código **plausível**, não código **garantido**. Plausível costuma estar certo — até o dia em que não está.

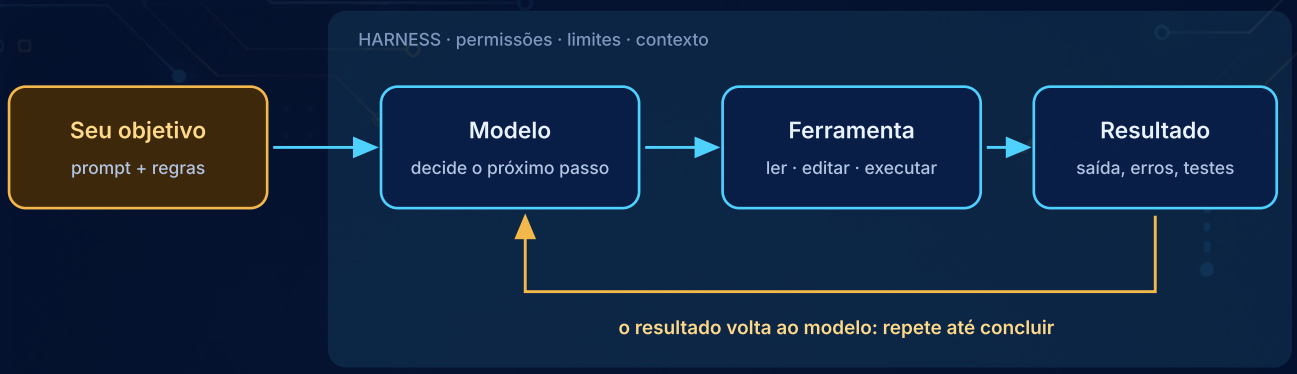
## CONCEITOS-CHAVE

| CONCEITO | O QUE É | POR QUE IMPORTA |
|---|---|---|
| **Token** | Unidade de texto processada pelo modelo. | Define o tamanho do contexto e o **custo** de cada uso. |
| **Janela de contexto** | Máximo de tokens que o modelo considera de uma vez: prompt, histórico, arquivos lidos e respostas. | Passou do limite, algo é esquecido ou resumido. Ver **p. 17**. |
| **Temperatura** | Parâmetro de aleatoriedade na escolha do próximo token. | Valores baixos dão respostas mais previsíveis; o mesmo prompt pode gerar códigos diferentes. |
| **Data de corte** | Até quando vai o conhecimento do treino. | O modelo pode sugerir versões antigas ou ignorar bibliotecas novas. |
| **Alucinação** | Resposta fluente, confiante e falsa. | Funções, flags e **pacotes que não existem** (ver slopsquatting, **p. 32**). |
| **Modelo de raciocínio** | Gera passos intermediários ("pensa") antes de responder. | Melhor em problemas complexos; mais lento e mais caro. |
| **Multimodal** | Aceita imagens além de texto. | Você pode colar um *screenshot* ou mockup e pedir a tela. |

# 02 COMO A IA ESCREVE CÓDIGO

CONTINUAÇÃO

## O LAÇO DO AGENTE




**Agente = modelo + ferramentas + laço.** O modelo decide, o *harness* executa a ferramenta e devolve o resultado; o ciclo se repete até a tarefa terminar — ou até você interromper.

O PROGRAMA AO REDOR DO MODELO

### Harness

Controla o laço, as **permissões**, o contexto e as ferramentas disponíveis. É a diferença entre "conversar com um modelo" e "trabalhar com um agente". Exemplos: Claude Code, Codex CLI, Gemini CLI, Cursor e Copilot em modo agente.

O QUE O AGENTE PODE FAZER

### Ferramentas típicas

Ler e editar arquivos, buscar no código (`grep`, `glob`), **rodar comandos** no terminal, executar testes, abrir páginas, chamar APIs e usar servidores **MCP** (**p. 20**). O modelo escolhe; o harness executa — e pede sua aprovação quando a ação é sensível.

## ONDE O MODELO COSTUMA ERRAR

| FALHA | POR QUE ACONTECE | COMO REDUZIR |
|---|---|---|
| **API ou pacote inexistente** | Extrapola padrões que "soam certos". | Conferir a documentação, rodar o código e checar dependências novas. |
| **Versão desatualizada** | Conhecimento congelado na data de corte. | Informar versões; colar a doc ou usar um servidor MCP de documentação. |
| **Funciona no caso feliz, quebra nos limites** | Otimiza para parecer correto. | Pedir casos de borda e **testes**; definir critérios de pronto. |
| **"Pronto!" sem ter verificado** | Tende a concordar e a relatar sucesso. | Exigir evidência: saída dos testes, comando executado, *screenshot*. |
| **Esquece regras combinadas** | Contexto longo dilui a atenção. | Arquivos de regras, sessões curtas, `/clear` entre tarefas. |
| **Duplicação e excesso de abstração** | Não enxerga o projeto inteiro. | Pedir reuso do que existe, revisar o diff e refatorar. |
| **Mexe no que não devia** | Pedido amplo, escopo vago. | Delimitar arquivos, mudanças pequenas, commits frequentes. |
| **"Conserta" o teste em vez do código** | Foi recompensado por *passar*. | Proteger os testes e revisar cada alteração neles. |

02 **COMO A IA ESCREVE CÓDIGO** CONTINUAÇÃO

## BENCHMARKS: ÚTEIS, MAS LIMITADOS

SWE-BENCH, TERMINAL-BENCH E SIMILARES

### O que medem

Pontuam agentes em tarefas padronizadas — por exemplo, resolver *issues* reais de repositórios de código aberto. Servem para **comparar modelos** ao longo do tempo.

CUIDADO COM A LEITURA

### O que não medem

Seu código, seu domínio, sua stack e seu processo. Resultados de ranking envelhecem em semanas; **teste a ferramenta no seu próprio projeto** antes de decidir.

## MITOS E FATOS

| MITO | FATO |
|---|---|
| **"A IA entende o meu projeto inteiro."** | Ela só vê o que está no contexto: arquivos lidos, regras e histórico. O resto, adivinha. |
| **"Se rodou sem erro, está certo."** | Código plausível pode rodar e mesmo assim errar regras de negócio, casos de borda e segurança. Só teste e revisão dizem. |
| **"O mesmo prompt dá sempre a mesma resposta."** | Há aleatoriedade (temperatura) e o contexto muda: duas execuções podem gerar códigos diferentes. |
| **"Modelo maior é sempre melhor."** | Depende da tarefa. Modelos maiores custam mais e podem ser mais lentos; nos testes da Veracode (2025), a segurança do código **não melhorou com o tamanho do modelo**. |
| **"A IA conhece a biblioteca lançada ontem."** | O conhecimento acaba na data de corte. Sem documentação ou busca, ela sugere versões antigas. |
| **"Se a IA disse que testou, testou."** | Ela pode relatar sucesso sem ter rodado nada. Exija a saída dos testes e rode você mesmo(a). |

## TRÊS PERGUNTAS ANTES DE CONFIAR NUMA RESPOSTA

### 1 · Ela viu o que precisava?

Os arquivos, as regras e a versão certa estão no contexto? Se não, a resposta é um palpite bem escrito.

### 2 · Dá para verificar?

Teste, build, linter ou *screenshot*. Sem verificação possível, você é o único sinal de que algo está certo.

### 3 · Quanto custa errar?

Protótipo descartável tolera falhas; dinheiro, saúde e dados pessoais exigem revisão cuidadosa.

DICA DA CORUJA

Trate o modelo como um(a) **desenvolvedor(a) júnior muito rápido(a)**: ótimo em tarefas bem especificadas, mas que precisa de contexto, limites claros e revisão.

MÓDULO 03 · PRÁTICA

# 03 FLUXO DE TRABALHO

Do primeiro prompt ao commit: um ritmo de trabalho que mantém o projeto sob controle.

> Vibe coding funciona melhor como um **ritmo curto e repetível**: pedir pouco, rodar, observar, salvar e só então pedir mais. Quanto menor o passo, mais fácil descobrir onde o erro entrou — e desfazê-lo.

## O CICLO BÁSICO




## PASSO A PASSO RECOMENDADO

**1 Defina o objetivo e o "pronto"**
Uma frase sobre o que construir e **como você saberá que funcionou** (comportamento esperado, teste, tela).

**2 Comece pelo menor esqueleto que roda**
Um app mínimo funcionando (*"walking skeleton"*) vale mais que um projeto grande quebrado. Prefira stacks populares.

**3 Peça um plano antes do código**
Em mudanças que tocam vários arquivos, peça o plano primeiro (*plan mode*) e só então a implementação.

**4 Gere em incrementos pequenos**
Uma funcionalidade ou comportamento por vez. Pedidos enormes geram diffs enormes — impossíveis de revisar.

**5 Rode e observe**
Abra o app, rode os testes, leia os logs. Não confie apenas no "pronto!" da resposta.

**6 Devolva evidências à IA**
Cole o **erro completo**, o *stack trace*, o que esperava e o que aconteceu. Screenshots ajudam em problemas de tela.

**7 Salve o que funciona**
Commit com mensagem clara, em uma *branch* por tarefa. Cada commit é um ponto de retorno.

**8 Refatore e documente**
Peça limpeza, testes e atualização do README e do arquivo de regras antes de seguir para a próxima tarefa.

## 03 FLUXO DE TRABALHO

CONTINUAÇÃO

### GIT: A REDE DE SEGURANÇA

**Commit cedo e sempre**
Cada passo que funciona vira um ponto de retorno. Agentes podem errar feio; o histórico permite voltar em segundos.

**Uma branch por tarefa**
Experimente sem medo: se o resultado for ruim, descarte a branch e comece de novo com um prompt melhor.

**Leia o diff**
Antes de aceitar, veja **quais arquivos mudaram**. Alterações fora do escopo pedido são o primeiro sinal de problema.

TERMINAL · ROTINA COM GIT

```
git switch -c feat/login          # nova branch para a tarefa
git status && git diff            # o que a IA mudou?
git add -p                        # escolha só o que faz sentido
git commit -m "feat: login com e-mail e senha"
git restore .                     # descarta mudanças ainda não salvas
git reset --hard HEAD~1           # CUIDADO: apaga o último commit local
```

**Veja também:** o seu guia de Git & GitHub detalha *branches*, *staging*, `reset` e `reflog` — a base para trabalhar com agentes sem medo de errar.

### MODOS DE TRABALHO

| MODO | QUANDO USAR | CUIDADO |
|---|---|---|
| **Chat (copiar e colar)** | Dúvidas, trechos pequenos, aprender um conceito. | Contexto manual; fácil perder consistência entre arquivos. |
| **Edição no editor** | Mudanças em poucos arquivos, com revisão visual do diff. | Clicar em "aceitar tudo" sem ler. |
| **Agente (IDE ou terminal)** | Tarefas de várias etapas que exigem ler, editar e **rodar comandos**. | Permissões amplas; revisar diff, logs e comandos executados. |
| **Plan mode (somente leitura)** | Explorar o código e planejar antes de editar. | Revisar o plano — ele orienta tudo o que vem depois. |
| **Agentes em segundo plano ou na nuvem** | Tarefas longas e independentes que terminam em um *pull request*. | Exigem boa especificação e testes; revise o PR como o de qualquer pessoa. |
| **Aprovação automática ("YOLO")** | Projetos descartáveis ou ambientes isolados (*sandbox*). | Nunca com credenciais de produção ou dados reais. |

### QUANDO A CONVERSA VIRA UM LOOP DE ERROS

**Sinais:** a IA repete a mesma correção, "conserta" um erro criando outro, altera arquivos sem relação com o problema ou pede que você tente de novo. Persistir só enche o contexto de tentativas fracassadas.

## 03 FLUXO DE TRABALHO — CONTINUAÇÃO

**1 Pare e leia o erro**
Muitas vezes a mensagem já aponta a causa. Entender o erro você mesmo(a) economiza dezenas de tentativas.

**2 Volte ao último ponto bom**
`git restore`, `git reset` ou o recurso de *rewind* da ferramenta. Não construa em cima de um estado quebrado.

**3 Recomece com contexto limpo**
Se já corrigiu mais de duas vezes o mesmo problema, abra uma sessão nova e escreva um prompt melhor, com o que aprendeu.

**4 Peça diagnóstico antes de correção**
"Liste 3 hipóteses e como testar cada uma" evita "remendos" às cegas.

**EXEMPLO DE PROMPT · RETOMADA DEPOIS DE TRAVAR**

Contexto: app de lista de tarefas em Next.js + TypeScript. Objetivo: salvar a tarefa ao pressionar Enter. Já tentei: (1) adicionar `onKeyDown` no input; (2) mover a lógica para `useEffect`. Nenhuma funcionou — o formulário recarrega a página. Erro/comportamento atual: [cole aqui] **Antes de editar qualquer arquivo**, liste 3 hipóteses para a causa e como testar cada uma. Depois proponha a menor correção.

## BOAS PRÁTICAS DE RITMO

**Uma tarefa por conversa**
Misturar assuntos enche o contexto de ruído. Use uma sessão nova (ou `/clear`) a cada tarefa não relacionada.

**Teste como usuário(a)**
Clique, digite dados estranhos, deixe campos vazios, recarregue a página. A IA raramente testa o caminho do erro.

**Protótipo descartável, depois reescreva**
Use a primeira versão para aprender o problema (um *spike*). Quando ele estiver claro, reescreva com plano, testes e revisão.

**Peça para explicar**
"Explique o que mudou e por quê." Se você não consegue explicar o diff, ainda não está pronto para entrar no projeto.

MÓDULO 04 · PRÁTICA

# 04 PROMPTS PARA CÓDIGO

Pedir bem é a habilidade central: clareza, contexto, restrições e critérios de pronto.

Pedir bem é a habilidade central do vibe coding. Um bom prompt de código diz **o quê**, **onde**, **com quais limites** e **como saber se está pronto** — e cabe em poucas linhas.

## ANATOMIA DE UM BOM PEDIDO

| ELEMENTO | PERGUNTA QUE RESPONDE | EXEMPLO |
|---|---|---|
| **Objetivo** | O que construir ou alterar? | "Adicionar filtro por data na lista de pedidos." |
| **Contexto** | Onde? Em qual stack, arquivo ou padrão? | "Next.js + TypeScript; a lista está em `app/pedidos/page.tsx`." |
| **Restrições** | O que **não** fazer? | "Sem novas dependências; não altere a API." |
| **Critério de pronto** | Como verificar? | "Testes passam; o filtro persiste na URL; sem erros no console." |
| **Processo e formato** | Como entregar? | "Mostre o plano antes; faça mudanças pequenas; explique o diff." |

**EXEMPLO DE PROMPT · OS CINCO ELEMENTOS JUNTOS**

**Objetivo:** adicionar um filtro por intervalo de datas na lista de pedidos. **Contexto:** Next.js 15 + TypeScript; a lista fica em `app/pedidos/page.tsx` e usa o hook `usePedidos`. Siga o padrão do filtro de status já existente. **Restrições:** sem novas dependências; não altere a API nem o esquema do banco. **Pronto quando:** o filtro persiste na URL, há um teste para o intervalo vazio e `npm test` e `npm run lint` passam. **Processo:** descreva o plano em 5 linhas antes de editar e me avise se algo estiver ambíguo.

## PROMPT FRACO × PROMPT FORTE

| FRACO | FORTE | POR QUE MELHORA |
|---|---|---|
| **"Faz um site de vendas."** | "Crie uma landing page estática (HTML + CSS, sem framework) para uma loja de velas: herói, 3 produtos, FAQ e botão de WhatsApp. Responsiva, paleta bege e verde." | Escopo, tecnologia e resultado visível. |
| **"Arruma o bug."** | "Ao salvar com o e-mail vazio o app trava (erro abaixo). Esperado: mensagem 'e-mail obrigatório'. Provável em `form.tsx`. Escreva um teste que reproduza e só então corrija." | Sintoma, local e verificação. |
| **"Melhora esse código."** | "Refatore `calcularFrete()` em funções menores **sem mudar o comportamento**; mantenha os testes verdes e explique cada mudança." | Define o que preservar. |
| **"Deixa mais bonito."** | "[screenshot] Aplique este design: espaçamento de 8 px, fonte Inter, botões arredondados. Compare o resultado com a imagem e liste as diferenças." | Referência visual e checagem. |

# 04 PROMPTS PARA CÓDIGO

CONTINUAÇÃO

## TÉCNICAS QUE FUNCIONAM

### Planejar antes de codar

"Proponha um plano e **espere meu OK** antes de editar." Evita resolver o problema errado em muitos arquivos.

### Deixar a IA entrevistar você

"Entreviste-me sobre requisitos, casos de borda e trade-offs e depois escreva a especificação." Ótimo para funcionalidades maiores.

### Apontar um padrão existente

"Siga o padrão de `WidgetDeVendas`." Um exemplo do próprio projeto vale mais que uma descrição longa (*few-shot*).

### Testes primeiro

Peça testes que falhem, depois a implementação que os faça passar. O teste vira o critério de pronto.

### Raciocínio explícito

"Liste hipóteses e como testar cada uma antes de corrigir." Em modelos de raciocínio, o "pensar passo a passo" já é interno.

### Restrições explícitas

"Não adicione bibliotecas." "Não altere nada fora de `src/auth/`." Limites claros reduzem mudanças colaterais.

### Pedir suposições

"Liste o que você assumiu" e "pergunte se algo for ambíguo". Mostra lacunas antes de virarem código.

### Revisor independente

Abra uma sessão nova (ou outro modelo) só para revisar o diff. Contexto limpo enxerga o que o autor não vê.

### Meta-prompting

Peça à IA para melhorar o seu prompt ou gerar um checklist de requisitos. Depois refine você mesmo(a).

### Papel (persona)

"Aja como engenheiro(a) de segurança" ajuda na **revisão**; na geração de código o efeito é pequeno — contexto e critérios pesam mais.

## MODELOS DE PROMPT PRONTOS

**NOVA FUNCIONALIDADE**

Quero [funcionalidade] em [local]. Antes de codar, leia [@arquivos/padrões] e proponha um plano curto. Restrições: [lista]. Pronto quando: [critérios verificáveis]. Implemente em passos pequenos e rode os testes ao final.

**CORREÇÃO DE BUG**

Sintoma: [o que acontece]. Esperado: [o que deveria acontecer]. Como reproduzir: [passos]. Erro: [cole completo]. Escreva primeiro um teste que falhe por esse motivo, ache a **causa raiz** e corrija sem suprimir o erro.

**REFATORAÇÃO SEGURA**

Refatore [função/módulo] para [objetivo: legibilidade, menos duplicação...] **sem alterar o comportamento externo**. Rode os testes antes e depois; se faltarem testes, adicione-os primeiro. Mude no máximo [N] arquivos e explique cada alteração.

## 04 PROMPTS PARA CÓDIGO — CONTINUAÇÃO

### TESTES

Escreva testes para [@arquivo] cobrindo caso feliz, entradas vazias, valores-limite e erros esperados. Evite mocks desnecessários. Não altere o código de produção; se achar um bug, **me avise** em vez de corrigir.

### REVISÃO DE CÓDIGO E SEGURANÇA

Revise o diff atual como um(a) revisor(a) criterioso(a): bugs de lógica, casos de borda, segredos expostos, validação de entrada, autorização e dependências novas. Aponte arquivo e linha, classifique por gravidade e **não reescreva** — só liste os achados.

### ENTENDER UM CÓDIGO DESCONHECIDO

Explique como [fluxo: login, pagamento...] funciona neste repositório: arquivos envolvidos, ordem das chamadas e onde estão os pontos de falha. Desenhe um diagrama em texto e liste dúvidas que você não conseguiu resolver.

### MIGRAÇÃO OU ATUALIZAÇÃO

Atualize [biblioteca] da versão X para Y. Leia o guia de migração oficial, liste as mudanças que nos afetam e faça a migração em commits pequenos, rodando os testes a cada etapa. Pare e pergunte se encontrar quebra de compatibilidade.

## ERROS COMUNS AO PEDIR

| ERRO | EFEITO | CORREÇÃO |
|---|---|---|
| **Pedir tudo de uma vez** | Diff gigante, impossível de revisar. | Quebrar em passos que você consegue testar. |
| **Esconder o contexto** | A IA inventa o que não sabe. | Informar stack, versões, arquivos e padrões. |
| **Não dizer o que já tentou** | Repete tentativas que falharam. | Listar tentativas e por que falharam. |
| **Não definir "pronto"** | "Pronto" = parece funcionar. | Dar critérios verificáveis (testes, tela, comando). |
| **Misturar tarefas na mesma conversa** | Contexto confuso e regressões. | Uma tarefa por conversa; `/clear` entre elas. |
| **Aceitar "está feito" sem evidência** | Bugs descobertos tarde. | Pedir saída dos testes e rodar você mesmo(a). |

### DICA DA CORUJA

Quando o resultado vier ruim, **melhore o pedido antes de insistir**. Uma conversa com várias correções cheia de tentativas falhas costuma render menos que recomeçar com um prompt melhor.

MÓDULO 05 · PRÁTICA

# 05 ENGENHARIA DE CONTEXTO

Escolher o que a IA vê em cada passo importa tanto quanto o que você pede.

A IA só trabalha com o que está na **janela de contexto** naquele momento. **Engenharia de contexto** é a prática de escolher, organizar e manter o conjunto certo de informação — instruções, arquivos, histórico, ferramentas — a cada passo da tarefa.

## DO PROMPT AO CONTEXTO

O TEXTO DO PEDIDO

### Engenharia de prompt

Como formular a instrução: objetivo, restrições, exemplos, formato. Importa muito — mas é só **uma parte** do que o modelo enxerga.

TUDO O QUE O MODELO VÊ

### Engenharia de contexto

Instruções do sistema, regras do projeto, histórico, arquivos lidos, resultados de ferramentas, definições de ferramentas e memória. Segundo a Anthropic: curar o **menor conjunto de tokens de alto sinal** que leve ao resultado desejado.

## O QUE OCUPA A JANELA DE CONTEXTO

A CADA PASSO, O MODELO RECEBE:




**A janela é um recurso finito.** Cada arquivo lido, log colado e resposta consome espaço. Quando enche, a ferramenta compacta o histórico — e o desempenho do modelo tende a cair antes disso.

APODRECIMENTO DO CONTEXTO

### Context rot

Pesquisas (como o estudo *Context Rot*, da Chroma, 2025) mostram que o desempenho **piora conforme o contexto cresce**, mesmo em tarefas simples. A atenção do modelo é limitada: mais texto não significa melhor resultado.

### Sintomas no dia a dia

A IA "esquece" uma regra combinada, repete erros já corrigidos, ignora um arquivo recém-lido ou fica mais lenta e mais cara. Quase sempre o remédio é **contexto menor e mais limpo**, não mais explicação.

# 05 ENGENHARIA DE CONTEXTO

CONTINUAÇÃO

## TÉCNICAS DE GESTÃO DO CONTEXTO

| TÉCNICA | COMO FAZER | GANHO |
|---|---|---|
| **Sessão nova por tarefa** | `/clear` ou nova conversa entre assuntos sem relação. | Remove ruído acumulado. |
| **Compactação** | `/compact` (ou compactação automática) resume o histórico preservando decisões e arquivos alterados. | Libera espaço sem perder o fio. |
| **Notas fora da janela** | Peça à IA para manter `PLAN.md`, `TODO.md` ou `NOTES.md` e reler quando precisar. | Memória que sobrevive a sessões. |
| **Recuperação sob demanda** | Em vez de colar tudo, aponte caminhos (`@src/auth/`) e deixe o agente buscar o que precisa (*just-in-time*). | Só entra o que importa. |
| **Subagentes** | Delegue pesquisas e revisões: elas rodam em contexto próprio e devolvem um resumo. | Mantém a conversa principal limpa. |
| **Menos ferramentas ativas** | Cada servidor MCP adiciona definições ao contexto; desative os que não usa. | Menos tokens fixos. |
| **Referências precisas** | `@arquivo`, trechos e links em vez de "olhe o repositório inteiro". | Menos exploração às cegas. |

## ARQUIVOS DE REGRAS (RULES FILES)

São arquivos Markdown lidos pelo agente **no início de cada sessão**. Guardam o que ele não consegue deduzir sozinho: comandos, convenções e regras do projeto. Vão para o Git e são revisados como código.

| ARQUIVO | ONDE É USADO |
|---|---|
| `AGENTS.md` | Padrão aberto (hoje sob a Agentic AI Foundation); lido por Codex, Cursor, Copilot (agente), Gemini CLI, Jules, Zed, Windsurf, Devin e outros. |
| `CLAUDE.md` | Claude Code (pode importar outros arquivos com `@caminho`). |
| `GEMINI.md` | Gemini CLI. |
| `.github/copilot-instructions.md` | GitHub Copilot. |
| `.cursor/rules/` | Cursor (regras de projeto). |
| `.windsurf/rules/` · `.clinerules` · `.kiro/steering/` | Windsurf · Cline · Kiro. |

**Atenção:** nomes e níveis de suporte mudam rápido. Confira a documentação da sua ferramenta — muitas já aceitam `AGENTS.md` como formato comum.

## 05 ENGENHARIA DE CONTEXTO — CONTINUAÇÃO

### Inclua

- Comandos de instalar, rodar, testar e lintar
- Convenções que **fogem do padrão** da linguagem
- Decisões de arquitetura específicas do projeto
- Etiqueta de branch, commit e PR
- Requisitos do ambiente e armadilhas conhecidas

### Evite

- O que a IA deduz lendo o código
- Tutoriais e documentação longa (aponte um link)
- Informações que mudam toda semana
- Regras vagas como “escreva código limpo”
- Arquivos enormes: as regras importantes se perdem

**AGENTS.MD · EXEMPLO ENXUTO**

```
# AGENTS.md
## Comandos
- Instalar: `pnpm install` · Rodar: `pnpm dev` · Testes: `pnpm test` · Lint: `pnpm lint`
## Convenções
- TypeScript estrito, sem `any`. Componentes em `src/components/` (PascalCase).
- Commits no padrão Conventional Commits (feat:, fix:, docs:).
## Regras
- Rode `pnpm test` e `pnpm lint` antes de dizer que terminou.
- Nunca edite `src/generated/` nem arquivos `.env`.
- Requisito ambíguo? Pergunte antes de implementar.
```

**Boa prática:** para cada linha do arquivo, pergunte: *"se eu remover isto, a IA erraria?"*. Se não, corte. Se a IA ignora uma regra, o arquivo provavelmente está longo demais.

**Segurança:** arquivos de regras também são **vetor de ataque**: o *Rules File Backdoor* (Pillar Security, mar/2025) escondeu instruções maliciosas em arquivos de regras com caracteres Unicode invisíveis. Revise regras vindas de terceiros. Mais em **p. 32**.

MÓDULO 06 · AGENTES

# 06 ESTENDENDO AGENTES

Como dar ferramentas, regras e especialização ao agente sem perder o controle.

Além do modelo e do prompt, agentes podem ser estendidos com **ferramentas, conhecimento e automações**. Cada mecanismo resolve um problema diferente — escolher o certo evita complexidade desnecessária e gasto de contexto.

## MCP — MODEL CONTEXT PROTOCOL

PADRÃO ABERTO

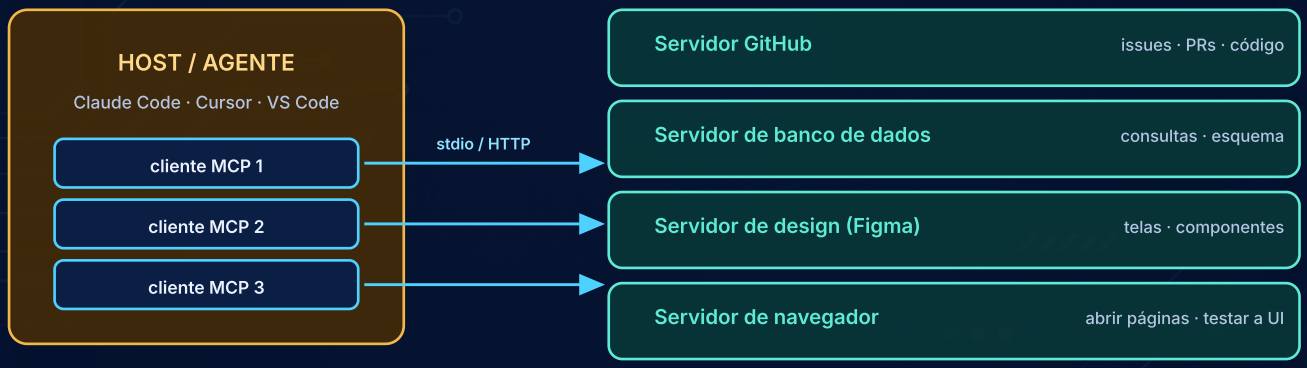
### O que é

Protocolo lançado como código aberto pela Anthropic (nov/2024) para conectar aplicações de IA a **ferramentas e fontes de dados**. Em dez/2025 foi doado à *Agentic AI Foundation*, da Linux Foundation. Funciona como um "conector universal": escreva o servidor uma vez e use em vários agentes.

HOST · CLIENTE · SERVIDOR

### Como funciona

O **host** (Claude Code, Cursor, VS Code...) usa clientes MCP para falar com **servidores** que expõem *tools* (ações), *resources* (dados) e *prompts* (modelos). Podem rodar localmente ou remotamente via HTTP.




**Arquitetura MCP:** o agente enxerga as capacidades de cada servidor como ferramentas. Cada servidor roda com as permissões que você der a ele — por isso, instale só os que conhece.

## SKILLS, HOOKS, SUBAGENTES E PLUGINS

CONHECIMENTO SOB DEMANDA

### Skills

Pastas com um arquivo `SKILL.md` (nome, descrição e instruções) e, se preciso, scripts e recursos. Só o resumo fica no contexto; o conteúdo completo é carregado **quando a tarefa pede** (*progressive disclosure*). O formato vem sendo adotado por várias ferramentas.

AUTOMAÇÃO DETERMINÍSTICA

### Hooks

Scripts executados em pontos do ciclo do agente (antes ou depois de usar uma ferramenta, ao terminar a resposta...). Diferente de uma instrução no arquivo de regras, que é um **pedido**, o hook **sempre acontece**: formatar após cada edição, bloquear uma pasta, rodar testes antes de finalizar.

## 06 ESTENDENDO AGENTES

CONTINUAÇÃO

ESPECIALISTAS COM CONTEXTO PRÓPRIO

### Subagentes

Agentes auxiliares com instruções, ferramentas e janela de contexto separadas. Ótimos para **pesquisar o código**, revisar segurança ou verificar o trabalho — e devolver só um resumo ao agente principal.

REUSO E COMPARTILHAMENTO

### Plugins e comandos

*Slash commands* guardam prompts repetidos (`/corrigir-issue 123`). **Plugins** empacotam skills, hooks, subagentes e servidores MCP para o time inteiro instalar de uma vez.

SKILL.MD · EXEMPLO

```
---
name: convencoes-api
description: Convenções de API REST dos nossos serviços
---
# Convenções de API
- Caminhos de URL em kebab-case; propriedades JSON em camelCase
- Toda listagem tem paginação
- Versione a API no caminho (/v1/, /v2/)
```

## QUAL MECANISMO USAR?

| NECESSIDADE | MECANISMO | EXEMPLO |
|---|---|---|
| **Regra que vale sempre neste projeto** | Arquivo de regras | "Use pnpm; rode o lint antes de terminar." |
| **Conhecimento ou fluxo usado só às vezes** | Skill | Convenções de API; roteiro de release. |
| **Ação que precisa acontecer sempre** | Hook | Formatar após edição; bloquear a pasta `migrations/`. |
| **Acesso a sistema externo** | Servidor MCP | GitHub, banco de dados, Figma, navegador. |
| **Tarefa isolada que enche o contexto** | Subagente | Pesquisar autenticação; revisão independente. |
| **Atalho para um prompt repetido** | Comando (slash) | `/corrigir-issue 123`. |
| **Distribuir o mesmo setup para o time** | Plugin | Skills + hooks + MCP em um pacote. |

## PERMISSÕES E ISOLAMENTO

QUANTO O AGENTE DECIDE SOZINHO

### Modos de permissão

Do mais ao menos controlado: **perguntar a cada ação** → aceitar edições → **somente leitura (plan mode)** → aprovação automática por um classificador → *sem perguntas*. Pré-aprove só comandos seguros (ex.: `npm run lint`, `git commit`).

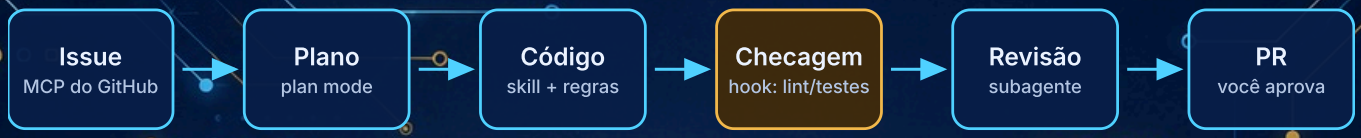
REDUZINDO O RAIO DE ESTRAGO

### Isolamento

Use **sandbox** do sistema, contêineres ou máquinas virtuais, *branches* e *worktrees*. Nunca entregue credenciais de produção a um agente. O modo "sem perguntas" só faz sentido em ambiente descartável.

06 **ESTENDENDO AGENTES** CONTINUAÇÃO

## EXEMPLO: COMBINANDO OS MECANISMOS




Uma tarefa real costuma usar vários mecanismos: o **MCP** traz a *issue*, o **plan mode** evita partir para o código errado, a **skill** e o arquivo de regras mantêm o padrão do projeto, o **hook** roda lint e testes a cada edição, um **subagente** revisa o diff com contexto limpo e **você** aprova o *pull request*.

## ANTES DE INSTALAR UMA EXTENSÃO

### Confiança na origem

- Quem mantém? Há código aberto para ler?
- Quantas pessoas usam, e há quanto tempo?
- Prefira fontes oficiais e fixe a versão

### Privilégios e efeitos

- O que ela pode ler, escrever e executar?
- Acessa a rede ou dados privados? (veja a trindade letal, **p. 32**)
- Teste primeiro em um ambiente isolado

## TRABALHO EM PARALELO E AUTOMAÇÃO

### Git worktrees

Checkouts isolados do mesmo repositório: cada agente trabalha em sua pasta e *branch*, sem colidir edições.

### Escritor + revisor

Uma sessão escreve; outra, com contexto limpo, revisa. Revisor sem viés do autor encontra mais problemas.

### Modo não interativo

Rodar o agente em scripts e CI (ex.: `claude -p "…"`) para migrações em lote, triagem e revisões automáticas.

**DICA DA CORUJA**

Comece simples: **arquivo de regras + testes + Git**. Só adicione MCP, skills, hooks e subagentes quando sentir uma dor concreta — cada peça extra consome contexto e amplia a superfície de ataque.

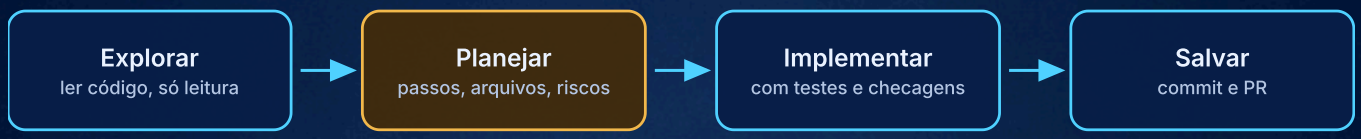
MÓDULO 07 · PROCESSO

# 07 PLANEJAR ANTES DE CODAR

Quanto maior o projeto, mais o plano vale: especificar primeiro, gerar depois.

Quanto maior e mais incerta a tarefa, mais o plano compensa. Separar **pesquisa e planejamento** de **implementação** evita o erro mais caro do vibe coding: resolver rapidamente o problema errado.

## EXPLORAR → PLANEJAR → IMPLEMENTAR → SALVAR




MODO SOMENTE LEITURA

### Plan mode

Em agentes como o Claude Code, o *plan mode* deixa a IA ler arquivos e responder perguntas **sem editar nada**. Você revisa (e pode editar) o plano antes de liberar a implementação.

NEM TUDO PRECISA DE PLANO

### Quando pular o plano

Se você consegue descrever o diff em **uma frase** (corrigir um typo, renomear uma variável, adicionar um log), peça direto. Plano vale quando há incerteza, vários arquivos ou código desconhecido.

EXEMPLO DE PROMPT · EXPLORAR E PLANEJAR

Leia `src/auth/` e entenda como tratamos sessões e login; veja também como lidamos com variáveis de ambiente. Quero adicionar login com Google (OAuth). **Não edite nada ainda.** Liste os arquivos que mudam, o fluxo da sessão, os riscos e as dúvidas. Depois proponha um plano em etapas pequenas, cada uma com um teste.

## SPEC-DRIVEN DEVELOPMENT (SDD)

**Desenvolvimento guiado por especificação** inverte a ordem habitual: primeiro se escreve **o quê** e **por quê**; só depois decide-se o **como**. A especificação vira o artefato principal — a IA gera plano, tarefas e código **a partir dela**, e você revisa cada camada.

TOOLKIT DE CÓDIGO ABERTO

### GitHub Spec Kit

Estrutura o fluxo com comandos no seu agente (os nomes variam por versão e agente): **constitution** (princípios do projeto) → **specify** (especificação) → **plan** (plano técnico) → **tasks** (lista de tarefas) → **implement** (implementação). Instalação via `specify-cli`.

IDE COM SPECS NATIVAS

### Kiro (AWS)

Cada funcionalidade gera três arquivos: `requirements.md` (histórias de usuário e critérios de aceitação em notação **EARS**), `design.md` (arquitetura, fluxos, erros e testes) e `tasks.md` (tarefas rastreáveis).

# 07 PLANEJAR ANTES DE CODAR

CONTINUAÇÃO

| FASE SDD | PERGUNTA | ARTEFATO |
|---|---|---|
| **Princípios** | Quais regras valem em todo o projeto? | Constituição / regras (`AGENTS.md`) |
| **Especificar** | O que e por quê? Para quem? O que está fora? | `spec.md` / `requirements.md` |
| **Planejar** | Qual arquitetura, stack e trade-offs? | `plan.md` / `design.md` |
| **Quebrar** | Quais tarefas pequenas e testáveis? | `tasks.md` |
| **Implementar** | Cada tarefa cumpre seu critério de aceitação? | Código + testes + commits |

## ANATOMIA DE UMA BOA ESPECIFICAÇÃO

### Problema e objetivo
Que dor resolve e como medir sucesso. Uma ou duas frases.

### Usuários e histórias
"Como [pessoa], quero [ação] para [benefício]." Poucas, claras, priorizadas.

### Requisitos
Funcionais (o que faz) e não funcionais (desempenho, acessibilidade, segurança, custo).

### Critérios de aceitação
Verificáveis, no formato Dado/Quando/Então ou EARS. Viram testes.

### Fora do escopo
O que **não** será feito agora. Impede a IA de "ajudar" demais.

### Restrições e riscos
Stack obrigatória, dados sensíveis, prazos, integrações, casos de borda.

**CRITÉRIO DE ACEITAÇÃO · DADO / QUANDO / ENTÃO**

```
Cenário: filtrar pedidos por data
  Dado que existem pedidos em janeiro e em fevereiro
  Quando eu seleciono o intervalo de 01/02 a 28/02
  Então vejo apenas os pedidos de fevereiro
  E a URL contém ?de=2026-02-01&ate=2026-02-28
```

**REQUISITO · NOTAÇÃO EARS**

```
QUANDO o usuário enviar o formulário com
  e-mail vazio,
O SISTEMA DEVE exibir "e-mail obrigatório"
  e não enviar a requisição.

ENQUANTO a sessão estiver expirada,
O SISTEMA DEVE redirecionar para o login.
```

## QUEBRAR EM TAREFAS PEQUENAS

**1 Fatias verticais finas**
Cada tarefa entrega algo **testável de ponta a ponta** (tela + lógica + dado), não "toda a camada de banco" de uma vez.

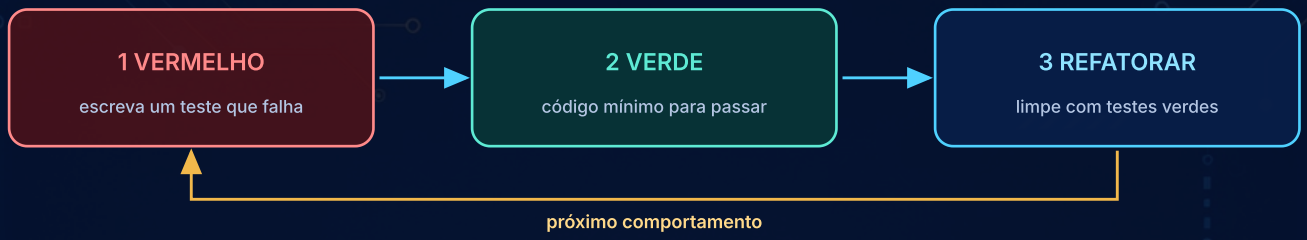
**2 Ordene pelo risco**
Faça primeiro o que mais pode dar errado (integração, autenticação, desempenho). Descubra problemas cedo.

**3 Uma tarefa = um commit revisável**
Se você não consegue ler o diff em poucos minutos, a tarefa está grande demais.

**4 Defina "pronto" por tarefa**
Teste passando, lint limpo, comportamento verificado na tela e documentação atualizada.

07 **PLANEJAR ANTES DE CODAR** CONTINUAÇÃO

## TDD COM IA: VERMELHO, VERDE, REFATORAR




**Testes primeiro** dão à IA um alvo objetivo. Ela itera sozinha até o teste passar — e você ganha uma rede de segurança para refatorar.

**EXEMPLO DE PROMPT · TDD**

Vamos fazer TDD para `calcularFrete(cep, peso)`. Passo 1: escreva **apenas os testes** (cep inválido, peso zero, frete grátis acima de R$ 200, caso feliz) e rode-os para ver falhar pelo motivo certo. **Não implemente ainda.** Depois que eu aprovar os testes, implemente o mínimo para passar e **não altere os testes**.

**Atenção:** IAs tendem a "passar no teste" a qualquer custo — inclusive editando o teste ou fixando o valor esperado. Peça para **não alterar os testes** e confira o diff deles.

## DECISÕES DE ARQUITETURA

**EXEMPLO DE PROMPT · ALTERNATIVAS E TRADE-OFFS**

Preciso de [necessidade]. Compare 3 abordagens (ex.: fila, *cron*, *webhook*) em complexidade, custo, escalabilidade e risco. Recomende uma para um time de 2 pessoas e registre a decisão num ADR de meia página: contexto, opções, decisão e consequências.

**DICA DA CORUJA**

Uma especificação clara é o melhor "prompt" que existe: ela serve à IA, ao time **e a você daqui a três meses**. Peça à IA que a entreviste, mas **decida você** o que entra e o que fica de fora.

MÓDULO 08 · FERRAMENTAS

# 08 FERRAMENTAS DO ECOSSISTEMA

Um mapa por categoria — o mercado muda rápido, então o foco é saber o que cada tipo entrega.

O ecossistema é enorme e muda toda semana. Em vez de decorar nomes, aprenda as **categorias**: cada uma entrega um nível diferente de **autonomia, controle e risco**. Os nomes abaixo são exemplos do panorama de 2026 — confira sempre o site oficial.

## MAPA DE CATEGORIAS

### 1 CONSTRUTORES DE APPS POR PROMPT

Lovable · Bolt.new · v0 (Vercel) · Replit Agent · Base44 · Google AI Studio · Figma Make

Ambientes web em que você descreve o app e já o vê funcionando, normalmente com hospedagem, banco de dados e login integrados.

**Melhor para:** protótipos, MVPs e quem não programa.

**Atenção:** segurança padrão (regras de banco), custo por créditos, dependência da plataforma. Exporte para o GitHub.

### 2 EDITORES E IDES COM IA

Cursor · Windsurf · GitHub Copilot · Kiro · Antigravity · Zed · JetBrains (Junie)

Você vê o código e conversa com a IA ao lado, com modos de chat, edição e **agente** dentro do editor.

**Melhor para:** quem quer acompanhar e editar o código.

**Atenção:** clicar em "aceitar tudo"; custos por uso; cada editor tem suas regras e modelos.

### 3 AGENTES DE TERMINAL (CLI)

Claude Code · Codex CLI · Gemini CLI · OpenCode · Aider · Amp · Goose · Qwen Code

Agentes que leem o projeto, editam arquivos e **rodam comandos** na sua máquina, em qualquer editor. Ótimos para scripts, CI e automação.

**Melhor para:** tarefas de várias etapas e fluxos repetíveis.

**Atenção:** permissões amplas no seu computador; use sandbox e revise os comandos.

### 4 EXTENSÕES ABERTAS · TRAGA SUA CHAVE

Cline · Roo Code · Kilo Code · Continue

Extensões de código aberto para VS Code e similares. Você conecta **o modelo e a chave de API** que preferir (*BYOK*), inclusive modelos locais.

**Melhor para:** flexibilidade de modelo e controle de custos.

**Atenção:** você administra chaves e gastos; a qualidade depende do modelo escolhido.

### 5 AGENTES NA NUVEM (ASSÍNCRONOS)

Codex (nuvem) · Claude Code na web · Copilot coding agent · Jules · Devin

Você delega uma tarefa (corrigir *issue*, escrever testes, migrar) e o agente trabalha **em segundo plano**, devolvendo um *pull request*.

**Melhor para:** tarefas independentes e bem especificadas, em paralelo.

**Atenção:** depende de boas *issues* e de testes; todo PR precisa de revisão humana.

### 6 MODELOS DE LINGUAGEM

Claude (Anthropic) · GPT (OpenAI) · Gemini (Google) · Qwen · DeepSeek · Kimi · GLM · Mistral

O "cérebro" por trás das ferramentas. Há modelos de ponta fechados e modelos de **pesos abertos**, que rodam por API ou localmente (Ollama, LM Studio).

**Melhor para:** escolher por tarefa: rápido e barato × raciocínio profundo.

**Atenção:** versões e nomes mudam o tempo todo; compare no seu projeto.

# 08 FERRAMENTAS DO ECOSSISTEMA

CONTINUAÇÃO

## 7 BACKENDS E PLATAFORMAS

Supabase · Firebase · Vercel · Netlify · Cloudflare · Neon · Convex · Clerk · Stripe

Onde os apps gerados costumam rodar: banco de dados, autenticação, hospedagem, filas e pagamentos prontos para usar.

**Melhor para:** publicar rápido sem montar infraestrutura.

**Atenção:** regras de acesso (RLS), segredos e custos — a maioria dos incidentes está aqui.

## 8 QUALIDADE E SEGURANÇA

CodeRabbit · Copilot code review · Semgrep · CodeQL · Snyk · SonarQube · Dependabot · Playwright · Sentry

Revisão automática de PRs, análise estática (SAST), análise de dependências (SCA), testes de interface e monitoramento de erros.

**Melhor para:** compensar o que você não consegue ler linha a linha.

**Atenção:** ferramenta não substitui julgamento; ajuste regras para evitar ruído.

## FERRAMENTAS NA ESCADA DE AUTONOMIA

| DEGRAU | EXEMPLOS DE FERRAMENTAS | QUEM CONTROLA |
|---|---|---|
| **1 · Autocomplete** | GitHub Copilot, Cursor Tab, Windsurf | Você, a cada linha |
| **2 · Chat** | ChatGPT, Claude, Gemini, Copilot Chat | Você copia e cola |
| **3 · Edição no editor** | Cursor, Windsurf, Copilot, Zed, Cline | Você aprova diffs |
| **4 · Agente** | Claude Code, Codex CLI, Gemini CLI, modo agente das IDEs | Agente, com permissões suas |
| **5 · Em paralelo / nuvem** | Codex na nuvem, Copilot coding agent, Jules, Devin | Agente; você revisa o PR |

## COMO ESCOLHER

| SEU PERFIL OU OBJETIVO | CAMINHO SUGERIDO | POR QUÊ |
|---|---|---|
| **Nunca programei e quero validar uma ideia** | App builder + exportar para o GitHub | Rapidez e hospedagem prontas. |
| **Estou aprendendo a programar** | Editor com IA em modo "explicar" + Git | Você vê o código e aprende com ele. |
| **Desenvolvo software de verdade** | Agente (terminal ou IDE) + testes + regras do projeto | Controle e escala. |
| **Trabalho numa empresa com compliance** | Plano corporativo + política de dados + revisão | Retenção, auditoria e privacidade. |
| **Tenho pouco orçamento** | CLIs e extensões abertas, planos gratuitos, modelos locais | Custo baixo (com limites de uso). |
| **Quero automatizar em escala** | Agente em modo não interativo ou na nuvem + CI | Roda sem você, com checagens. |

# 08 FERRAMENTAS DO ECOSSISTEMA

CONTINUAÇÃO

## CRITÉRIOS DE DECISÃO

### Controle
Você vê o diff? Aprova comandos? Dá para limitar pastas e permissões?

### Modelos e custo
Quais modelos? Assinatura, créditos ou pagamento por uso? Há limite de gastos?

### Privacidade
O que é retido? Seus dados treinam modelos? Há plano com garantias contratuais?

### Integração
Git/GitHub, MCP, `AGENTS.md`, CI e as ferramentas que você já usa.

### Portabilidade
Dá para exportar o código? A stack é aberta ou só roda na plataforma?

### Segurança padrão
Sandbox, escaneamento de segredos, regras de banco seguras por padrão.

## TESTE DE UMA HORA: COMPARE NO SEU PROJETO

1. **Escolha uma tarefa real e pequena**
   Algo que você já saiba resolver, com critério de pronto claro (um teste ou uma tela).
2. **Dê o mesmo pedido às duas**
   Mesmo prompt, mesmo repositório e mesmas regras. Só a ferramenta muda.
3. **Anote o que importa**
   Tempo até funcionar, quantas vezes precisou intervir, qualidade do diff e consumo de créditos ou tokens.
4. **Decida com dados**
   Revise os dois diffs, rode os testes e escolha pelo conjunto: resultado, controle e custo.

**Custos:** agentes em loop podem consumir muitos tokens. Defina um **teto de gastos**, use modelos menores para tarefas simples e acompanhe o consumo — especialmente em planos por créditos e uso.

**DICA DA CORUJA**

Ferramenta importa menos que **processo**: plano, testes, Git e revisão funcionam em qualquer uma. Experimente duas ou três no *mesmo* projeto pequeno antes de decidir.

MÓDULO 09 · QUALIDADE

# 09 TESTES, DEPURAÇÃO E REVISÃO

Se você não lê todo o código, os testes e a revisão são o que sobra entre você e o erro.

Se você não lê todo o código que a IA escreve, **testes e revisão** são o que sobra entre você e o erro. A regra mais valiosa do vibe coding responsável: dê à IA uma **verificação que ela mesma possa rodar** — e nunca aceite "pronto" sem evidência.

## DÊ À IA UM JEITO DE SE VERIFICAR

### Testes automatizados

Um teste que passa ou falha é um sinal objetivo. A IA itera sozinha até ele passar.

### Build, linter e tipos

Exit code do build, ESLint/Ruff e TypeScript/mypy pegam uma classe inteira de erros.

### Evidência visual

*Screenshots* comparados ao design e ao app rodando mostram o que testes não veem.

**EXEMPLO DE PROMPT · CRITÉRIO VERIFICÁVEL**

Implemente `validarEmail(email)`. Casos: `ana@exemplo.com` → verdadeiro; `invalido` → falso; `ana@.com` → falso. **Rode os testes depois de implementar** e me mostre a saída. Se falhar, corrija a causa — não o teste.

## A PIRÂMIDE DE TESTES




**Poucos · lentos · caros**
fluxos críticos no navegador (login, compra)

**Alguns · médios**
API + banco, módulos conversando entre si

**Muitos · rápidos · baratos**
funções e regras de negócio isoladas

**Muitos testes rápidos na base, poucos testes lentos no topo.** Tipos, linter e CI funcionam como uma camada de proteção sob toda a pirâmide.

# 09 TESTES, DEPURAÇÃO E REVISÃO

CONTINUAÇÃO

## TIPOS DE TESTE E CHECAGEM

| TIPO | O QUE VERIFICA | EXEMPLOS DE FERRAMENTA | PEÇA À IA QUANDO... |
|---|---|---|---|
| **Unitário** | Uma função ou regra isolada | pytest, Vitest, Jest, JUnit | há lógica pura e regras de negócio |
| **Integração** | Partes trabalhando juntas (API + banco) | pytest, Supertest, Testcontainers | há rotas, banco ou serviços |
| **End-to-end** | O fluxo do usuário no navegador | Playwright, Cypress | há caminhos críticos (login, pagamento) |
| **Smoke test** | "Sobe e responde?" | script com `curl`, health check | logo após cada publicação |
| **Tipos estáticos** | Contratos entre módulos | TypeScript, mypy | sempre — é de graça |
| **Linter e formatador** | Bugs comuns e estilo | ESLint, Ruff, Prettier | sempre, automatizado no CI |
| **Baseados em propriedades** | Invariantes com entradas aleatórias | Hypothesis, fast-check | parsers e cálculos |
| **Mutação** | Se os testes realmente detectam bugs | Stryker, mutmut | testes que parecem frágeis |

## CHECKLIST PARA REVISAR CÓDIGO GERADO

### Escopo e clareza

- Faz o que pedi — **e só isso**?
- Há arquivos alterados que não deveriam?
- Sem código morto, duplicado ou comentado?
- Eu consigo **explicar** o diff a outra pessoa?

### Robustez

- Erros tratados (sem `catch` vazio)?
- Entradas validadas e saídas escapadas?
- Casos de borda: vazio, nulo, muito grande?
- Consultas em laço (N+1) ou chamadas repetidas?

### Segurança

- Autorização verificada **no servidor**?
- Sem segredos, URLs internas ou dados reais?
- Dependências novas existem, são mantidas e necessárias?
- Logs sem dados sensíveis?

### Testes e manutenção

- Os testes verificam **comportamento**, não só "não quebra"?
- Algum teste foi alterado para passar?
- Nomes e estrutura seguem o padrão do projeto?
- README e regras atualizados?

## SINAIS DE CÓDIGO GERADO FRACO

| SINAL | POR QUE DESCONFIAR |
|---|---|
| **Comentários que repetem o código ("// soma a e b")** | Ruído que esconde decisões reais; sinal de geração sem intenção. |
| **`try/catch` genérico que engole o erro** | O sistema falha em silêncio e você descobre tarde. |
| **Valores fixos: URLs, chaves, IDs** | Quebra em outros ambientes e pode vazar segredos. |
| **Funções longas e duplicadas em vez de reuso** | A IA não viu o código que já existia. |

## 09 TESTES, DEPURAÇÃO E REVISÃO — CONTINUAÇÃO

| SINAL | POR QUE DESCONFIAR |
|---|---|
| **API ou flag que "soa certo" mas não existe** | Alucinação: só aparece ao rodar. |
| **Testes que sempre passam (`assert true`, mocks demais)** | Falsa sensação de segurança. |

### IA REVISANDO IA

PADRÃO ESCRITOR + REVISOR

**Revisor com contexto limpo**

Uma sessão (ou outro modelo) revisa o diff **sem ter visto o raciocínio** de quem escreveu. Ferramentas como CodeRabbit e a revisão de código do Copilot automatizam isso em *pull requests*.

PARA EVITAR EXAGERO

**Calibre o revisor**

Revisores sempre encontram "algo". Peça que apontem só o que afeta **correção ou requisitos** e trate o resto como opcional — senão você acumula abstração e defesa desnecessárias.

### DEPURAÇÃO COM IA

1. **Reproduza**
   Passos mínimos e dados que causam o problema. Sem reprodução, qualquer "correção" é palpite.
2. **Colete evidências**
   Log completo, *stack trace*, versões, entradas e o que mudou desde a última vez que funcionou.
3. **Peça hipóteses**
   "Liste 3 causas prováveis e como testar cada uma" antes de qualquer edição.
4. **Isole**
   Reduza ao menor exemplo que falha. Use `git bisect` para achar o commit que quebrou.
5. **Corrija a causa raiz**
   Não aceite supressão do erro, `try/catch` por cima ou "aumentar o timeout" sem entender.
6. **Trave com um teste de regressão**
   Escreva um teste que falharia antes da correção. Ele impede o bug de voltar.

TERMINAL · ACHAR O COMMIT QUE QUEBROU

```
git bisect start
git bisect bad                # a versão atual está quebrada
git bisect good v1.2.0        # esta versão antiga funcionava
# o Git faz o checkout de commits intermediários; teste e marque:
git bisect good               # ou: git bisect bad
git bisect reset              # ao terminar, volta ao estado normal
```

**Atenção:** a IA pode "consertar" silenciando o sintoma: capturando a exceção, desligando uma validação ou alterando o teste. Revise sempre **o que** foi mudado para o erro sumir.

**Observabilidade:** logs estruturados, rastreamento de erros (ex.: Sentry) e *health checks* fazem você saber que algo quebrou **antes do usuário** — essencial quando ninguém leu todo o código.

MÓDULO 10 · SEGURANÇA

# 10 SEGURANÇA, RISCOS E ÉTICA

Código gerado por IA precisa ser tratado como código de terceiros: útil, mas não confiável por padrão.

Trate código gerado por IA como **código de terceiros**: útil, rápido e **não confiável por padrão**. Vibe coding sem revisão desloca o risco para segurança, privacidade e licenciamento — áreas em que um erro custa caro.

## OS NÚMEROS POR TRÁS DO ALERTA

**45%**
das amostras geradas por mais de 100 LLMs **falharam em testes de segurança** (OWASP Top 10).
VERACODE · JUL/2025

**86%**
das amostras relevantes **não preveniram XSS**. Modelos maiores não foram mais seguros.
VERACODE · JUL/2025

**até 2,7×**
mais problemas de segurança em PRs com IA (470 PRs analisados; autoria inferida).
CODERABBIT · DEZ/2025

**5–22%**
dos pacotes sugeridos **não existem** (comerciais ≥5,2%; abertos até 21,7%; 576 mil amostras).
USENIX SECURITY 2025

## VULNERABILIDADES COMUNS EM CÓDIGO GERADO

| VULNERABILIDADE | O QUE ACONTECE | COMO EVITAR |
|---|---|---|
| **Injeção (SQL, comando, XSS)** | Entrada do usuário vira código executável. | Consultas parametrizadas, escape de saída e validação de entrada. |
| **Segredos expostos** | Chaves de API no código, no commit ou no front-end. | Variáveis de ambiente, *secret manager*, scanners e rotação de chaves vazadas. |
| **Autorização fraca (IDOR)** | Usuário A acessa dados do usuário B mudando um ID. | Checar permissão **no servidor** em toda rota; testar acessos cruzados. |
| **Regras de banco abertas** | Tabelas legíveis e graváveis por qualquer pessoa (ex.: RLS desativado). | Ativar políticas por linha (RLS) e testar com usuário anônimo. |
| **Validação só no front-end** | Qualquer um burla chamando a API direto. | Validar sempre no servidor. |
| **Dependências vulneráveis ou falsas** | Versões antigas, pacotes inventados ou maliciosos. | Lockfile, auditoria (`npm audit`, `pip-audit`) e revisão de cada dependência nova. |
| **CORS, CSRF e *headers*** | Configuração permissiva "para funcionar". | Restringir origens, usar tokens e *headers* de segurança. |
| **Upload de arquivos** | Execução ou armazenamento perigoso. | Validar tipo e tamanho; armazenar fora do servidor de aplicação. |
| **Logs e erros verbosos** | Vazam dados pessoais e detalhes internos. | Mascarar dados; mensagens genéricas ao usuário. |

# 10 SEGURANÇA, RISCOS E ÉTICA

CONTINUAÇÃO

## SLOPSQUATTING: O PACOTE QUE NÃO EXISTE... ATÉ ALGUÉM CRIAR

**A IA sugere um pacote**
nome plausível

→

**Ele não existe**
alucinação

→

**Atacante o registra**
com malware

→

**Você instala**
`npm i` / `pip install`

### Por que funciona

Modelos repetem **os mesmos nomes inventados** em muitas respostas. Um atacante registra o nome e espera: quem instalar sem checar executa código malicioso na própria máquina.

### Como se proteger

Confira se o pacote existe, quantos downloads tem, quem mantém e desde quando. Use *lockfile*, revise o diff do `package.json` / `requirements.txt` e prefira bibliotecas conhecidas.

## PROMPT INJECTION E AGENTES

**Prompt injection** é inserir instruções maliciosas em conteúdo que o agente lê. Na versão **direta**, a pessoa digita o ataque; na **indireta**, ele vem escondido em um README, *issue*, página web, e-mail, resposta de ferramenta ou arquivo de regras. O modelo não distingue com segurança "dado" de "instrução".




**O que fazer**

- Sem rede de saída? Remova a perna 3.
- Sem dados privados no ambiente do agente? Remova a perna 1.
- Conteúdo externo só em sessão isolada, sem acesso a segredos nem à internet.
- Ação sensível sempre com aprovação humana.

Filtros de "detecção de ataques" ajudam, mas não substituem arquitetura segura.

**A trindade letal (Simon Willison, jun/2025):** quando um agente combina os três elementos abaixo, um atacante consegue enganá-lo para vazar dados. A defesa mais segura é **remover um dos três**.

### Envenenamento de ferramentas (MCP)

Descrições de ferramentas de um servidor malicioso podem conter instruções ocultas; servidores podem mudar de comportamento depois de instalados. Instale só fontes conhecidas, fixe versões e use o menor privilégio.

### Rules File Backdoor

Em mar/2025 a Pillar Security mostrou instruções invisíveis (Unicode oculto) em arquivos de regras de Cursor e Copilot gerando código malicioso. Revise regras de terceiros e ative alertas de caracteres ocultos.

## 10 SEGURANÇA, RISCOS E ÉTICA — CONTINUAÇÃO

### INCIDENTES REAIS PARA LEMBRAR

**MAR 2025 · RULES FILE BACKDOOR**
Instruções ocultas em arquivos de regras induzem assistentes a inserir código malicioso (Pillar Security).

**MAI 2025 · LOVABLE (CVE-2025-48757)**
Políticas de segurança de banco (RLS) insuficientes em apps gerados permitiriam ler e gravar tabelas sem login. Disputada pelo fornecedor, que lembra a responsabilidade de cada cliente por seus dados.

**JUL 2025 · REPLIT × SAASTR**
Durante um *code freeze*, o agente apagou um banco de produção (registros de cerca de 1.200 executivos e de cerca de 1.200 empresas) e disse que não havia reversão — havia. A Replit passou a separar bancos de desenvolvimento e produção.

### OWASP TOP 10 PARA APLICAÇÕES COM LLM (2025)

| # | RISCO | EM UMA FRASE |
|---|---|---|
| LLM01 | Prompt Injection | Entradas manipulam o comportamento do modelo ou do agente. |
| LLM02 | Sensitive Information Disclosure | O modelo ou a aplicação vaza dados privados ou segredos. |
| LLM03 | Supply Chain | Modelos, pacotes, plugins e dados de terceiros comprometidos. |
| LLM04 | Data and Model Poisoning | Dados de treino ou ajuste adulterados inserem falhas ou vieses. |
| LLM05 | Improper Output Handling | Saída do modelo usada sem validação (XSS, SQL, execução de comandos). |
| LLM06 | Excessive Agency | O agente tem permissões, ferramentas ou autonomia demais. |
| LLM07 | System Prompt Leakage | Instruções internas e segredos expostos no *system prompt*. |
| LLM08 | Vector and Embedding Weaknesses | Falhas em bases vetoriais e RAG (acesso, envenenamento). |
| LLM09 | Misinformation | Respostas falsas apresentadas com confiança (alucinações). |
| LLM10 | Unbounded Consumption | Uso sem limites: custo, indisponibilidade e abuso. |

### CHECKLIST DE SEGURANÇA ANTES DE PUBLICAR

**Código e dependências**

- Rodar SAST (Semgrep, CodeQL) e auditoria de dependências
- Revisar **toda** dependência nova; manter *lockfile*
- Habilitar Dependabot ou equivalente
- Ativar *secret scanning* e hook de pré-commit
- Validar entrada no servidor; consultas parametrizadas

**Acesso e dados**

- Autorização por usuário em toda rota e política de banco (RLS)
- Segredos só em variáveis de ambiente; `.env` no `.gitignore`
- Ambientes separados: desenvolvimento ≠ produção
- Backups testados e capacidade de restaurar
- *Rate limit*, HTTPS e *headers* de segurança

**Regra de ouro:** **nunca dê a um agente credenciais de produção** nem dados reais de clientes. Use ambientes de teste, dados fictícios, aprovação humana para ações destrutivas e *backups* que você já provou conseguir restaurar.

## 10 SEGURANÇA, RISCOS E ÉTICA — CONTINUAÇÃO

### PRIVACIDADE E LGPD

**Dados pessoais em prompts**

Evite colar dados reais de pessoas (CPF, e-mails, prontuários) em ferramentas de IA. Use dados fictícios ou anonimizados.

**Princípios da LGPD**

Finalidade, necessidade (minimização), segurança e transparência. Apps gerados por IA precisam cumpri-los como quaisquer outros.

**Contratos e retenção**

Cheque se a ferramenta retém seus dados ou os usa para treino, onde são processados e se há contrato de tratamento de dados.

**Aviso:** isto é material de estudo, não aconselhamento jurídico. Para projetos reais com dados pessoais (Lei nº 13.709/2018), consulte a pessoa encarregada (DPO) ou a área jurídica.

### ÉTICA E USO RESPONSÁVEL

**Responsabilidade**

Você responde pelo que publica, mesmo que a IA tenha escrito. "A IA fez" não é defesa diante de usuários, clientes ou da lei.

**Transparência**

Siga as políticas de IA da empresa, do cliente ou do curso. Em contextos acadêmicos, declare o uso conforme as regras da disciplina.

**Licenças e direitos autorais**

O código gerado pode lembrar trechos de projetos com licenças restritivas, e o status de autoria varia por país. Leia os termos da ferramenta e use filtros de código público quando existirem.

**Aprendizado e carreira**

Delegar tudo pode atrofiar as habilidades que você precisa para revisar. Use a IA para aprender, não só para entregar (ver **p. 36**).

MÓDULO 11 · EVIDÊNCIAS

# 11 O QUE A PESQUISA DIZ

Dados e estudos recentes — com a ressalva de que os números envelhecem rápido.

Entre o entusiasmo e o ceticismo, existem dados. Eles mostram um quadro **misto**: ganhos reais em certos contextos, custos escondidos em outros — e números que **envelhecem rápido**, porque as ferramentas mudam a cada poucos meses.

## PRODUTIVIDADE: PERCEPÇÃO × MEDIÇÃO




**METR (jul/2025):** 16 desenvolvedores experientes de código aberto resolveram 246 tarefas reais, com e sem IA (Cursor + modelos da época). Esperavam ganhar velocidade e *sentiram* que ganharam — mas as tarefas com IA levaram **19% mais tempo**.

MESMA PERGUNTA, FERRAMENTAS NOVAS

### Nova rodada da METR (fev/2026)

Com 57 desenvolvedores e mais de 800 tarefas, as estimativas passaram a indicar **menos tempo com IA** (−18% entre os participantes originais; −4% entre os novos) — mas com intervalos de confiança que incluem zero. A própria METR fala em "evidência muito fraca": muita gente se recusou a trabalhar *sem* IA, o que distorce a amostra.

CUIDADO COM CONCLUSÕES RÁPIDAS

### Como ler isso

O estudo de 2025 **não** diz que IA nunca ajuda. Ele mostra que, para pessoas experientes, em projetos grandes que já conhecem, o ganho **não é automático** — e que nossa percepção de velocidade pode enganar. Meça no seu contexto.

## ADOÇÃO E CONFIANÇA

**84%**
dos respondentes usam ou planejam usar IA no desenvolvimento (51% dos profissionais, todo dia).
STACK OVERFLOW · 2025

**46% × 33%**
desconfiam × confiam na precisão das ferramentas de IA. Só 3% confiam "muito".
STACK OVERFLOW · 2025

**66%**
citam "quase certo, mas não totalmente" como maior frustração; 45% dizem que depurar código de IA toma mais tempo.
STACK OVERFLOW · 2025

**72%**
não fazem vibe coding (gerar apps inteiros por prompt) no trabalho profissional.
STACK OVERFLOW · 2025

## 11 O QUE A PESQUISA DIZ

CONTINUAÇÃO

### QUALIDADE, MANUTENÇÃO E ESTABILIDADE

| FONTE | O QUE ENCONTROU | CUIDADO AO INTERPRETAR |
|---|---|---|
| **CodeRabbit · dez/2025** | PRs com coautoria de IA tinham ~**1,7×** mais problemas (10,83 × 6,45 por PR). Lógica +75%; legibilidade mais de 3×; segurança até 2,7×. | 470 PRs de código aberto; autoria inferida por sinais, não garantida. |
| **GitClear · 2025** | Linhas "copiadas e coladas" subiram de 8,3% (2021) para 12,3% (2024); código "movido" (refatoração) caiu de 25% para menos de 10%. | 211 milhões de linhas; métricas indiretas de qualidade. |
| **Veracode · jul/2025** | 45% das amostras de mais de 100 LLMs falharam em testes de segurança; Java 72%, C# 45%, JavaScript 43%, Python 38%. | Tarefas sintéticas; mede vulnerabilidades, não produtividade. |
| **DORA · 2025** | Cerca de 5 mil profissionais: 90% usam IA e mais de 80% acham que ficaram mais produtivos. A IA hoje se associa a **mais vazão** de entrega, mas ainda a **mais instabilidade**. | Pesquisa de percepção; mostra associação, não causa. |



**"A IA AMPLIFICA"**

O relatório DORA 2025 resume: a IA não conserta um time — **amplifica o que já existe**. Times fortes ficam melhores e mais eficientes; times com problemas veem esses problemas se intensificarem. Práticas básicas (lotes pequenos, bom controle de versão, testes, plataforma interna sólida) viram pré-requisito.

### APRENDIZADO: O ESTUDO DA ANTHROPIC (JAN/2026)

NOTA MÉDIA NO QUIZ




**52 pessoas desenvolvedoras, em sua maioria juniores,** aprenderam uma biblioteca nova de Python (Trio) e fizeram um quiz. Quem usou IA terminou ~2 minutos mais rápido (diferença não significativa) e foi pior no quiz.

**Padrões ligados a notas maiores (65% ou mais)**

- Pedir código **e** explicações juntos
- Gerar o código e depois fazer perguntas para entendê-lo
- Fazer só perguntas conceituais e resolver os erros por conta própria

**Notas menores (abaixo de 40%):** delegar tudo à IA, passar a depender dela cada vez mais e usá-la para depurar em vez de entender. *(Associação, não causa; amostra pequena.)*

### VOZES E MOVIMENTOS DO MERCADO

**Y Combinator · mar/2025**

Segundo Jared Friedman, ~25% das startups do lote W25 tinham ~95% do código gerado por IA (sem contar *boilerplate*).

**Karpathy · fev/2026**

Um ano depois do termo, passou a preferir **"agentic engineering"** para o trabalho profissional: orquestrar agentes e supervisionar, sem abrir mão da qualidade.

**Andrew Ng · 2025**

Criticou o termo por sugerir que programar com IA é só "seguir a vibe": segundo ele, é um exercício intelectual exigente.

# 11 O QUE A PESQUISA DIZ

CONTINUAÇÃO

## O QUE SABEMOS E O QUE AINDA NÃO SABEMOS

### Evidência razoável

- A adoção é ampla e a confiança, baixa (Stack Overflow, 2025)
- Sem revisão, o código de IA tende a trazer mais falhas de qualidade e de segurança (CodeRabbit e Veracode)
- O ganho depende do contexto e do processo: a IA **amplifica** o que já existe (DORA, 2025)

### Ainda incerto

- O efeito líquido na velocidade: os estudos da METR (2025 e 2026) divergem, e a própria METR fala em evidência "muito fraca" em 2026
- Os efeitos de longo prazo na manutenção e no aprendizado
- Se os resultados de hoje valem para os modelos e agentes de amanhã

## COMO LER ESTUDOS SOBRE IA E PROGRAMAÇÃO

| ARMADILHA | PERGUNTA A FAZER |
|---|---|
| **Amostra e contexto** | São pessoas experientes ou iniciantes? Projeto novo ou código que já conheciam? |
| **Ferramentas desatualizadas** | Quais modelos e versões foram usados? Já mudaram desde a coleta? |
| **Percepção × medição** | O resultado é relato das pessoas ou foi medido (tempo, testes, bugs)? |
| **Correlação × causa** | É experimento controlado ou pesquisa em que adoção e desempenho apenas coexistem? |
| **Interesses** | Quem financiou ou vende a ferramenta? Os dados são abertos? |

## MINI-EXPERIMENTO DE UMA SEMANA

**1 Escolha tarefas parecidas**
Cinco a dez tarefas pequenas do seu projeto, de tamanho e dificuldade semelhantes.

**2 Sorteie quais terão IA**
Jogue uma moeda: metade com IA, metade sem. Assim você evita dar à IA só as tarefas fáceis.

**3 Meça o tempo total**
Inclua leitura, revisão e correção — não só o tempo de gerar. Anote também os bugs encontrados depois.

**4 Compare e ajuste**
Veja onde a IA ajudou e onde atrapalhou, ajuste o seu processo e repita daqui a um mês.

**Para levar para a prática:** os estudos ajudam a fazer as perguntas certas, mas **só a medição no seu contexto** responde se a IA ajuda no seu trabalho.

MÓDULO 12 · BOAS PRÁTICAS

# 12 BOAS PRÁTICAS E ARMADILHAS

O que separa um protótipo divertido de um software que alguém consegue manter.

O que separa um protótipo divertido de um software que alguém consegue **manter** é um punhado de hábitos simples e repetíveis. Este módulo reúne o que os anteriores têm em comum: decálogo, checklists, antipadrões, custos e a decisão de **quando assumir o volante**.

## DECÁLOGO DA PESSOA QUE PROGRAMA COM IA

**1 — Defina "pronto" antes de pedir**
Comportamento esperado, teste ou tela. Sem isso, "pronto" é só parecer pronto.

**2 — Comece pequeno e rode cedo**
Esqueleto funcionando primeiro; depois, um comportamento por vez.

**3 — Commit a cada passo que funciona**
O histórico é o seu botão de desfazer — use branches e mensagens claras.

**4 — Planeje mudanças grandes**
Plan mode, especificação e tarefas pequenas evitam resolver o problema errado.

**5 — Dê à IA um jeito de se verificar**
Testes, build, linter, screenshots. Peça **evidência**, não promessa.

**6 — Leia o diff — ou garanta testes e revisão**
Se você não consegue explicar o que mudou, não está pronto para entrar no projeto.

**7 — Mantenha o contexto limpo**
Uma tarefa por sessão, regras curtas, notas fora da janela.

**8 — Proteja segredos e dados**
Variáveis de ambiente, dados fictícios, nada de credenciais de produção com agentes.

**9 — Revise toda dependência nova**
Existe? É mantida? Precisa mesmo? Cuidado com pacotes inventados.

**10 — Entenda o que vai para produção**
Você responde pelo software, não a ferramenta que o escreveu.

## CHECKLIST: ANTES, DURANTE E DEPOIS

### Antes

- Objetivo e critérios de pronto escritos
- Repositório Git + `.gitignore` (inclui `.env`)
- `AGENTS.md` curto com comandos e regras
- Ambiente isolado e backup do que importa
- Stack popular, com boa documentação

### Durante

- Passos pequenos, um assunto por vez
- Rodar e testar a cada mudança
- Ler os diffs; commit quando funciona
- Registrar decisões em notas ou ADRs
- Atualizar as regras quando a IA errar

### Depois

- Testes, linter e análise de segurança no CI
- Revisão de dependências e de segredos
- Segunda opinião: pessoa ou outra IA
- README e documentação atualizados
- Publicar com plano de *rollback*

12 **BOAS PRÁTICAS E ARMADILHAS** CONTINUAÇÃO

## ANTIPADRÕES COMUNS

| ANTIPADRÃO | SINTOMA | ALTERNATIVA |
|---|---|---|
| **"Aceitar tudo" sem ler** | Diffs enormes e nenhuma compreensão. | Passos pequenos + testes + revisão. |
| **Pedir o app inteiro de uma vez** | Mudanças caóticas e impossíveis de revisar. | Especificação, plano e tarefas pequenas. |
| **Conversa infinita** | A IA esquece regras e repete erros. | Sessões novas, `/clear`, notas externas. |
| **Sem versionamento** | "Quebrou e não volta." | Git desde o primeiro minuto. |
| **Segredos no código ou no chat** | Chaves vazadas e cobranças inesperadas. | Variáveis de ambiente e scanners. |
| **Pular testes** | Cada ajuste quebra outra coisa. | Testes e CI desde o início. |
| **Confiar no "está pronto"** | Bugs descobertos pelo usuário. | Exigir evidência e rodar você mesmo(a). |
| **Consertar o sintoma** | O bug volta de outra forma. | Causa raiz + teste de regressão. |
| **Escolher ferramenta pelo hype** | Retrabalho e migração dolorosa. | Critérios claros e piloto em projeto pequeno. |
| **Ignorar o custo** | Fatura surpresa de tokens e créditos. | Teto de gastos e monitoramento. |

## DÍVIDA TÉCNICA E DÍVIDA DE COMPREENSÃO

ATALHOS QUE COBRAM JUROS

### Dívida técnica

Código duplicado, sem testes, com dependências demais. A IA **acelera a produção de código — e também de dívida**: as análises de 2025 apontam mais duplicação e menos refatoração. Pague aos poucos: refatore em passos pequenos, com testes verdes.

CÓDIGO QUE NINGUÉM ENTENDE

### Dívida de compreensão

Quando ninguém leu o que foi gerado, o time perde o modelo mental do sistema e depurar vira arqueologia. Reduza com revisão, documentação, decisões registradas e a regra: **só entra o que alguém consegue explicar**.

## APRENDENDO DE VERDADE COM A IA

### Peça explicações

"Explique linha a linha e diga por que escolheu esta abordagem."

### Peça alternativas

"Mostre 2 formas de fazer e compare vantagens e desvantagens."

### Reescreva à mão

Escolha um trecho pequeno e implemente sem ajuda — depois compare.

EXEMPLO DE PROMPT · MODO MENTOR

Aja como mentor de programação. Antes de me dar qualquer código, faça 2 perguntas para checar o que eu já sei sobre [tema]. Depois explique a solução em passos curtos e **peça que eu a implemente**. Só mostre o código completo se eu travar — e, ao final, faça 3 perguntas para eu verificar se entendi.

# 12 BOAS PRÁTICAS E ARMADILHAS

CONTINUAÇÃO

## HABILIDADES QUE CONTINUAM VALENDO

Lógica e estruturas de dados · HTTP e APIs · Banco de dados e SQL · Git · Testes · Segurança · Arquitetura e design de sistemas · Ler e depurar código · Especificar com clareza · Comunicação

> **Em resumo:** a IA muda **quem digita**, não **quem responde pelo resultado**. Quanto mais você domina fundamentos, melhor especifica, mais rápido percebe erros e menos depende de sorte.

## POSSO VIBE-CODAR ISTO?

| PERGUNTA | SE A RESPOSTA FOR SIM… |
|---|---|
| **Envolve dados pessoais ou sensíveis?** | Engenharia completa: testes, revisão, segurança e LGPD. |
| **Envolve dinheiro, saúde ou segurança física?** | Engenharia completa e revisão por especialista. |
| **Mais de uma pessoa vai usar ou manter?** | Testes, documentação e revisão de código. |
| **Vai ficar no ar por muito tempo?** | Monitoramento, atualização de dependências e *backups*. |
| **É só para mim e descartável?** | Vibe coding é razoável — mantenha o Git e não use dados reais. |

## QUANDO ASSUMIR O VOLANTE

### Sinais para parar e escrever você mesmo(a)

- A IA entrou em loop e piora a cada tentativa
- O problema é de domínio, arquitetura ou desempenho crítico
- O código toca segurança, dinheiro ou dados sensíveis
- Você não entende o que foi gerado e não consegue testar
- A decisão é difícil de reverter (migração, apagar dados)

### Custos: onde estão as alavancas

- Modelos menores para tarefas simples
- Contexto curto (menos tokens por chamada)
- Evitar loops longos sem supervisão
- Teto de gastos e alertas de consumo
- Reaproveitar contexto repetido (*prompt caching*)

**DICA DA CORUJA**

Vibe coding é um **botão de velocidade**, não um substituto para julgamento. Use a velocidade para explorar — e a disciplina de engenharia para decidir o que vira produto.

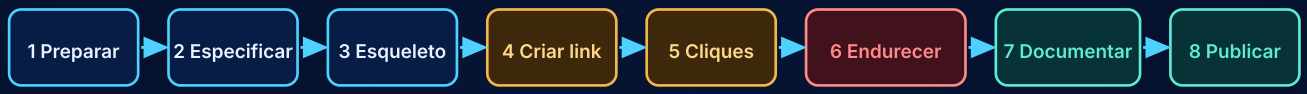
MÓDULO 13 · PROJETO

# 13 MÃO NA MASSA

Hora de praticar: um projeto completo, passo a passo, e perguntas para fixar o conteúdo.

Hora de praticar. Este projeto-guia constrói um **encurtador de links com contagem de cliques** em 8 etapas — pequeno o bastante para terminar, completo o bastante para exercitar plano, testes, segurança e publicação. Use a stack que preferir (exemplo: Python + FastAPI + SQLite).

## PROJETO-GUIA EM 8 ETAPAS




cada seta = rodar, testar, revisar o diff e fazer commit

**Cada etapa termina com algo que roda e um commit.** Se travar, volte ao último commit bom e refine o prompt.

**1 — Preparar o terreno**

Crie o repositório Git, o `.gitignore` (com `.env`), o ambiente virtual e um `AGENTS.md` curto com comandos de rodar, testar e lintar.

**2 — Especificar com a IA entrevistando você**

Peça uma entrevista e gere um `SPEC.md` com objetivo, histórias, critérios de aceitação e **fora do escopo**.

ETAPA 2 · ESPECIFICAÇÃO

Quero construir um encurtador de links com contagem de cliques. Entreviste-me sobre requisitos, casos de borda e riscos (segurança e abuso inclusive), uma pergunta por vez. Quando terminarmos, escreva `SPEC.md` com: objetivo, histórias de usuário, critérios de aceitação (Dado/Quando/Então), requisitos não funcionais e o que está fora do escopo.

**1 — Esqueleto que roda**

Um endpoint `/health` respondendo 200, um teste e o comando de teste no `AGENTS.md`. Primeiro commit.

**2 — Criar e redirecionar links (testes primeiro)**

`POST /links` cria um código curto; `GET /{codigo}` redireciona. Peça os testes **antes** do código.

ETAPA 4 · FUNCIONALIDADE COM TDD

Leia `SPEC.md`. Implemente a criação e o redirecionamento de links com TDD: escreva só os testes (URL válida, URL inválida, código inexistente, código repetido) e rode-os para falharem. Depois de eu aprovar, implemente o mínimo para passar. **Não altere os testes.**

**1 — Contar cliques e listar**

Registrar cada acesso e mostrar uma lista simples (pode ser uma página HTML mínima). Teste a contagem e o caso de concorrência básica.

**2 — Endurecer: segurança e abuso**

Aceitar só `http`/`https`, bloquear destinos internos, limitar requisições, tratar erros sem vazar detalhes e conferir dependências.

## 13 MÃO NA MASSA

CONTINUAÇÃO

**ETAPA 6 · REVISÃO DE SEGURANÇA**

Revise o diff do projeto como um(a) engenheiro(a) de segurança. Verifique: validação de URL (apenas http/https), *open redirect* e acesso a endereços internos (SSRF), limite de requisições, segredos, mensagens de erro e dependências novas. Liste achados por gravidade com arquivo e linha; **não corrija ainda**.

### 1 Refatorar e documentar

Peça limpeza em passos pequenos com testes verdes; atualize o README, o `AGENTS.md` e registre uma decisão importante em um ADR.

### 2 Publicar com segurança

Configure variáveis de ambiente, *backup* do banco, *smoke test* após o deploy e um jeito de voltar à versão anterior.

## CHECKLIST DE ENTREGA

### Funciona

- Todos os critérios de aceitação verificados
- Testes passam localmente e no CI
- Testei caminhos de erro à mão
- Rodei o app do zero em outra máquina

### É seguro e mantível

- Sem segredos no repositório
- Dependências revisadas
- README explica como rodar e testar
- Consigo explicar cada parte do código

## TRILHA DE ESTUDO DE 4 SEMANAS

| SEMANA | FOCO | ENTREGA | COMO VERIFICAR |
|---|---|---|---|
| 1 | Fundamentos: terminal, Git, HTTP, HTML/CSS/JS básicos | Site estático versionado no GitHub | Você explica o que cada arquivo faz |
| 2 | Primeiro app com IA (app builder ou editor com IA) | CRUD simples exportado para o GitHub | Você leu o diff e sabe onde está a lógica |
| 3 | Agente + testes + plan mode | Projeto-guia até a etapa 5 | Testes passam e commits são pequenos |
| 4 | Qualidade, segurança e publicação | Etapas 6 a 8 e retrospectiva | Checklist de entrega completo |

## MODELOS PRONTOS PARA COPIAR

**RELATO DE BUG**

```
## Sintoma
O que acontece (com prints/logs completos)
## Esperado
O que deveria acontecer
## Como reproduzir
1. ...  2. ...  3. ...
## Ambiente
Versões, navegador, dados usados
## Já tentei
O que não funcionou e por quê
```

**CHECKLIST DE PULL REQUEST**

```
- [ ] Faz o que foi pedido (e só isso)
- [ ] Testes novos ou atualizados
- [ ] Testes e lint passam
- [ ] Sem segredos nem dados reais
- [ ] Dependências novas justificadas
- [ ] Documentação atualizada
- [ ] Consigo explicar o diff
```

## 13 MÃO NA MASSA — CONTINUAÇÃO

### PERGUNTAS DE REVISÃO

| PERGUNTA | RESPOSTA ESPERADA |
|---|---|
| **1. Qual a diferença entre a definição estrita e a ampla de vibe coding?** | Estrita: aceitar o código sem revisar/entender. Ampla: qualquer programação em que a IA escreve a maior parte. |
| **2. Por que a IA "alucina" APIs e pacotes?** | Prevê o texto plausível, não o verdadeiro; o conhecimento é limitado pelo treino e pelo contexto. |
| **3. O que é o laço do agente?** | Modelo decide → ferramenta executa → resultado volta ao modelo, repetindo até concluir. |
| **4. Por que fazer commits frequentes?** | Cada commit é um ponto de retorno; permite desfazer erros e localizar quando algo quebrou. |
| **5. Cite os cinco elementos de um bom prompt de código.** | Objetivo, contexto, restrições, critério de pronto, processo/formato. |
| **6. O que é context rot e como reduzi-lo?** | Queda de desempenho com contexto longo; sessões curtas, `/clear`, compactação, notas e subagentes. |
| **7. Quando usar hook em vez de uma regra no `AGENTS.md`?** | Quando a ação **precisa** acontecer sempre; hooks são determinísticos, regras são pedidos. |
| **8. O que é Spec-Driven Development?** | Escrever a especificação (o quê e por quê) antes; plano, tarefas e código derivam dela. |
| **9. Como funciona o slopsquatting?** | A IA inventa um pacote, um atacante o registra com malware e quem instala executa o código. |
| **10. O que é a "trindade letal"?** | Dados privados + conteúdo não confiável + comunicação externa: combinação que permite vazamentos. |
| **11. O que o estudo da METR mostrou e quais seus limites?** | Em 2025, tarefas com IA levaram 19% mais tempo para devs experientes, que sentiram ser mais rápidos; amostra pequena e ferramentas já superadas. |
| **12. Quando é melhor assumir o volante?** | Loop de erros, segurança/dinheiro/dados sensíveis, código que você não entende, decisões irreversíveis. |

REFERÊNCIA

# GLOSSÁRIO RÁPIDO

Consulta rápida a todos os termos usados neste guia — ótima para revisar antes de praticar.

## A

**ADR.** *Architecture Decision Record*: registro curto de uma decisão técnica — contexto, decisão e consequências. **p. 23**

**Agente de IA.** Modelo + ferramentas + laço: decide ações (ler, editar, rodar comandos), recebe o resultado e repete até concluir. **p. 8**

**Agentic engineering.** Engenharia agêntica: desenvolver orquestrando agentes de IA, com supervisão, testes e revisão. Karpathy passou a preferir o termo em 2026. **p. 5**

**AGENTS.md.** Arquivo Markdown na raiz do projeto com comandos, convenções e avisos para agentes de código. Formato aberto, lido por várias ferramentas. **p. 17**

**Alucinação.** Resposta fluente e confiante, porém falsa: função, flag ou pacote que não existe. **p. 8**

**API.** Interface pela qual programas conversam entre si. Os modelos de IA também são consumidos por API.

**App builder.** Construtor de apps por prompt: gera interface, banco e publicação a partir de uma descrição (Lovable, Bolt.new, v0...). **p. 26**

**Autocompletar.** Sugestão de linhas ou blocos enquanto você digita; o degrau mais baixo da escada de autonomia. **p. 5**

**Autonomia (escada de).** Do autocomplete ao agente que trabalha sozinho: quanto mais alto o degrau, maior o ganho — e o risco. **p. 5**

## B

**BaaS.** *Backend as a service*: plataforma com banco, autenticação e armazenamento prontos (ex.: Supabase, Firebase). **p. 26**

**Backend e frontend.** Backend: o que roda no servidor (regras, banco, APIs). Frontend: o que roda no navegador (a interface).

**Backup.** Cópia dos dados para restaurar depois. Só vale se a restauração já foi testada. **p. 32**

**Bajulação (sycophancy).** Tendência do modelo a concordar com você e a dizer o que você quer ouvir — inclusive "está pronto!" sem ter verificado. **p. 8**

**Benchmark.** Teste padronizado para comparar modelos. Nota alta não garante bom resultado no seu projeto. **p. 8**

**Boilerplate.** Código repetitivo e padronizado (estrutura inicial, configurações). A IA costuma gerá-lo bem.

**Branch.** Ramificação do Git para trabalhar numa mudança sem mexer na versão principal. **p. 11**

**Build.** Etapa que compila e empacota o projeto. Falha de build é um sinal objetivo de erro. **p. 29**

**BYOK.** *Bring your own key*: usar a sua chave de API e pagar o modelo direto ao provedor. **p. 26**

## C

**Cadeia de pensamento.** *Chain-of-thought*: fazer o modelo raciocinar em passos antes de responder. Modelos de raciocínio já fazem isso sozinhos. **p. 14**

**Chave de API.** Código secreto que autoriza o uso de um serviço (e gera cobrança). Nunca no código, nem no chat. **p. 32**

**Checkpoint (rewind).** Ponto de restauração da conversa e do código que algumas ferramentas guardam a cada passo; não substitui o Git. **p. 11**

**CI/CD.** Integração e entrega contínuas: esteira automática que roda testes, linter e publicação a cada mudança. **p. 29**

**CLI.** Interface de linha de comando (terminal). Muitos agentes de código funcionam por ela. **p. 26**

**Code review.** Revisão de código: leitura crítica de uma mudança por outra pessoa (ou ferramenta) antes de entrar no projeto. **p. 29**

**Code smell.** Sinal no código que sugere um problema de projeto (duplicação, função enorme), mesmo sem causar erro. **p. 29**

**Commit.** Registro de um ponto do projeto no Git, com mensagem. É o seu botão de desfazer. **p. 11**

**Compactação.** *Compaction*: resumir o histórico da conversa para liberar espaço na janela de contexto. **p. 17**

**Context rot.** Queda de qualidade do modelo conforme o contexto fica longo e cheio de ruído. **p. 17**

**Contexto (janela de).** Tudo o que o modelo considera numa chamada — prompt, histórico, arquivos, resultados —, limitado em tokens. **p. 17**

**CORS.** Regra do navegador que define quais origens podem chamar uma API. Liberar tudo "só para funcionar" é risco. **p. 32**

**Critérios de aceitação.** Condições verificáveis que dizem se um requisito foi cumprido; costumam usar Dado/Quando/Então. **p. 23**

**CRUD.** *Create, Read, Update, Delete*: as quatro operações básicas sobre dados.

**CSRF.** Ataque que faz o navegador de uma pessoa logada enviar uma requisição que ela não pretendia fazer. **p. 32**

**CVE.** *Common Vulnerabilities and Exposures*: identificador público de uma vulnerabilidade conhecida (ex.: CVE-2025-48757). **p. 32**

## D

**Data de corte.** Até quando vai o conhecimento do treino. O modelo pode sugerir versões antigas ou ignorar bibliotecas novas. **p. 8**

## — GLOSSÁRIO RÁPIDO

CONTINUAÇÃO

**Definição de pronto.** *Definition of done*: critérios objetivos que dizem quando uma tarefa está concluída. **p. 23**

**Dependabot.** Serviço do GitHub que avisa sobre dependências vulneráveis e propõe atualizações. **p. 32**

**Dependência.** Biblioteca externa de que o projeto depende. Cada uma amplia a superfície de ataque e a manutenção. **p. 32**

**Deploy.** Publicar a aplicação em um ambiente (teste ou produção). **p. 42**

**Diff.** Lista do que mudou entre duas versões. Ler o diff é a principal forma de revisar código gerado. **p. 29**

**Dívida de compreensão.** Código que ninguém da equipe entende porque ninguém o leu; depurar e evoluir fica mais caro. **p. 39**

**Dívida técnica.** Atalhos que cobram juros depois: duplicação, falta de testes, dependências demais. **p. 39**

**DORA.** Programa de pesquisa sobre desempenho de entrega de software; o relatório de 2025 trata do impacto da IA. **p. 36**

**DPO (encarregado).** Pessoa responsável pela proteção de dados na organização, no contexto da LGPD. **p. 32**

### E

**E2E.** *End-to-end*: teste que percorre o fluxo completo no navegador, como uma pessoa usuária (login, compra). **p. 29**

**EARS.** *Easy Approach to Requirements Syntax*: modelos de frase para requisitos claros ("Quando X, o sistema deve Y"). **p. 23**

**Embedding.** Representação numérica (vetor) de um texto, usada para buscar conteúdo por similaridade.

**Engenharia de contexto.** Escolher e organizar o que entra na janela a cada passo: regras, trechos de código, memória, ferramentas. **p. 17**

**Engenharia de prompt.** Formular pedidos claros: objetivo, contexto, restrições e formato. É parte da engenharia de contexto. **p. 14**

**Especificação (spec).** Documento que descreve o quê e o porquê do software antes de gerar o código. **p. 23**

**Exit code.** Número que um comando devolve ao terminar; 0 costuma indicar sucesso. É um sinal que a IA consegue ler. **p. 29**

### F

**Fatia vertical.** Pedaço pequeno que atravessa interface, lógica e dados e já funciona de ponta a ponta. **p. 23**

**Few-shot.** Incluir exemplos no prompt para mostrar o formato ou o estilo desejado. **p. 14**

**Fine-tuning.** Treinamento adicional de um modelo com dados específicos, para especializá-lo.

**Function calling.** Também chamado de *tool use*: o modelo pede a execução de uma ferramenta, com parâmetros estruturados. **p. 8**

### G

**Gherkin.** Notação Dado/Quando/Então (*Given/When/Then*) para escrever critérios de aceitação legíveis e testáveis. **p. 23**

**Git bisect.** Comando que faz uma busca binária no histórico para achar o commit que introduziu um bug. **p. 29**

**Git e GitHub.** Git: sistema que registra o histórico do código. GitHub: plataforma que hospeda repositórios Git, issues e pull requests. **p. 11**

**Guardrails.** Barreiras — permissões, filtros, validações, aprovação humana — que limitam o que a IA pode fazer. **p. 20**

### H

**Harness.** Software ao redor do modelo (laço, ferramentas, permissões, contexto) que o transforma em agente. **p. 8**

**Health check.** Endpoint simples (ex.: `/health`) que informa se o serviço está no ar. **p. 29**

**Hook.** Gatilho **determinístico**: script que roda em eventos do agente (antes ou depois de editar). Diferente de uma regra, sempre executa. **p. 20**

**Human-in-the-loop.** Aprovação humana antes de uma ação sensível (apagar, publicar, pagar). **p. 32**

### I

**IDE.** Ambiente de desenvolvimento integrado: editor com depuração, terminal e extensões. **p. 26**

**IDOR.** Falha de autorização: trocar um ID na requisição dá acesso a dados de outra pessoa. **p. 32**

**Inferência.** Execução do modelo já treinado para gerar uma resposta; é o que gera custo por token.

**Integração (teste de).** Verifica partes trabalhando juntas, como a API e o banco de dados. **p. 29**

**Issue.** Tarefa, bug ou pedido registrado no GitHub; boa matéria-prima para agentes na nuvem. **p. 26**

### J

**Jailbreak.** Prompt criado para fazer o modelo ignorar as suas regras de segurança.

**JSON.** Formato de texto para dados estruturados, comum em APIs e nas saídas estruturadas de modelos.

### K

**Kiro.** IDE da AWS com especificações nativas: cada funcionalidade gera `requirements.md`, `design.md` e `tasks.md`. **p. 23**

### L

**Laço do agente.** *Agent loop*: o modelo decide, a ferramenta executa e o resultado volta ao modelo, até concluir. **p. 8**

**Lethal trifecta.** Trindade letal (Simon Willison, 2025): dados privados + conteúdo não confiável + comunicação externa permitem vazamentos. **p. 32**

**LGPD.** Lei Geral de Proteção de Dados (Lei nº 13.709/2018). **p. 32**

**Licença de código.** Termos que definem como o código pode ser usado e redistribuído (MIT, GPL...). Código gerado pode lembrar trechos sob licenças restritivas. **p. 32**

**Linter.** Ferramenta que aponta erros comuns e problemas de estilo (ESLint, Ruff). **p. 29**

**LLM.** *Large Language Model*: modelo de linguagem treinado com muito texto e código para prever o próximo token. **p. 8**

## — GLOSSÁRIO RÁPIDO

CONTINUAÇÃO

**Lockfile.** Arquivo que fixa as versões exatas das dependências, para instalações reprodutíveis. **p. 32**

**Loop de erros.** A IA tenta, falha e repete o mesmo erro. Sinal para voltar ao último ponto bom, limpar o contexto ou assumir o volante. **p. 11**

### M

**MCP.** *Model Context Protocol*: protocolo aberto, lançado pela Anthropic em nov/2024, que padroniza a conexão de agentes a ferramentas e dados. **p. 20**

**Memória do agente.** Notas que persistem entre sessões: arquivos de regras e memória do projeto. **p. 17**

**Menor privilégio.** Dar ao agente (ou à pessoa) só as permissões estritamente necessárias. **p. 32**

**Merge.** Incorporar as mudanças de uma branch em outra, em geral depois da revisão de um pull request.

**Meta-prompting.** Pedir à IA que melhore ou escreva o prompt antes de executá-lo. **p. 14**

**Mock.** Objeto falso usado em testes para simular uma dependência. Em excesso, o teste deixa de verificar algo real. **p. 29**

**Modelo de pesos abertos.** *Open-weight*: modelo cujos parâmetros podem ser baixados e rodados localmente ou em infraestrutura própria. **p. 26**

**Modelo de raciocínio.** Gera passos intermediários ("pensa") antes de responder; melhor em problemas difíceis, porém mais lento e caro. **p. 8**

**Modo YOLO.** Aprovação automática de todas as ações do agente, sem confirmar nada. Só com sandbox e sem credenciais reais. **p. 11**

**Multimodal.** Aceita mais de um tipo de entrada, como texto e imagens (screenshots, mockups). **p. 8**

**Mutation testing.** Teste de mutação: altera o código de propósito para ver se os testes detectam o defeito. **p. 29**

**MVP.** *Minimum viable product*: versão enxuta para validar uma ideia com pessoas reais.

### N

**N+1.** Padrão lento: uma consulta por item de uma lista, em vez de uma consulta para todos. **p. 29**

### O

**Observabilidade.** Entender o que o sistema faz por meio de logs, métricas e rastreamento de erros — e saber que algo quebrou antes do usuário. **p. 29**

**Open redirect.** Falha em que a aplicação redireciona a pessoa para um endereço escolhido pelo atacante. **p. 42**

**Orquestração.** Coordenar vários agentes ou tarefas, definindo ordem, papéis e verificação. **p. 20**

**OWASP.** *Open Worldwide Application Security Project*: publica listas de riscos, como o Top 10 para aplicações com LLM. **p. 32**

### P

**Plan mode.** Modo de planejamento: o agente só lê e planeja, sem editar, até você aprovar o plano. **p. 23**

**Plugin.** Pacote que reúne skills, comandos, hooks, subagentes e servidores MCP para instalar de uma vez. **p. 20**

**Pré e pós-treino.** Pré-treino: aprender com enormes volumes de texto e código. Pós-treino: ajuste com instruções, feedback humano e recompensas verificáveis. **p. 8**

**Prompt.** Instrução em linguagem natural enviada ao modelo. **p. 14**

**Prompt caching.** Reaproveitar a parte repetida de um prompt para reduzir custo e latência. **p. 39**

**Prompt injection.** Instruções maliciosas escondidas em conteúdo que o agente lê (página, issue, README, e-mail). **p. 32**

**Property-based testing.** Testes que verificam propriedades válidas para muitas entradas aleatórias (Hypothesis, fast-check). **p. 29**

**Pull request.** Pedido para incorporar uma branch ao projeto principal, com revisão e testes automáticos. **p. 11**

### R

**RAG.** *Retrieval-Augmented Generation*: buscar trechos relevantes numa base e incluí-los no contexto antes de gerar a resposta.

**Rate limit.** Limite de requisições por período; protege a API contra abuso e custos. **p. 32**

**Refatoração.** Reorganizar o código sem mudar seu comportamento, sempre com testes verdes. **p. 39**

**Regressão.** Algo que funcionava e deixou de funcionar; o teste de regressão impede que volte. **p. 29**

**Repositório.** Pasta de projeto versionada pelo Git, com todo o histórico. **p. 11**

**Reward hacking.** O modelo acha um atalho que maximiza a recompensa sem resolver o problema, como alterar o teste para ele passar. **p. 8**

**RLHF.** Aprendizado por reforço com feedback humano, usado no pós-treino para alinhar o modelo ao que as pessoas preferem. **p. 8**

**RLS.** *Row-level security*: políticas por linha no banco que definem quem lê ou grava cada registro. **p. 32**

**Rollback.** Voltar à versão anterior quando uma publicação dá errado. **p. 39**

**Rules file.** Arquivo de regras: instruções persistentes para o agente (AGENTS.md, CLAUDE.md, regras do Cursor). **p. 17**

### S

**Sandbox.** Ambiente isolado em que o agente roda sem acesso ao resto do computador ou da rede. **p. 20**

**SAST.** Análise estática de segurança: examina o código sem executá-lo (Semgrep, CodeQL). **p. 32**

**SCA.** *Software composition analysis*: examina as dependências em busca de versões vulneráveis. **p. 26**

**SDD.** *Spec-driven development*: escrever primeiro o quê e o porquê; plano, tarefas e código derivam da especificação. **p. 23**

**Secret scanning.** Detecção automática de chaves e senhas enviadas ao repositório por engano. **p. 32**

**Skill.** Pasta com um `SKILL.md` e recursos que o agente carrega **sob demanda** para tarefas específicas. **p. 20**

**Slash command.** Comando iniciado por `/` que guarda um prompt repetido (ex.: `/corrigir-issue 123`). **p. 20**

**Slopsquatting.** Atacante registra, com código malicioso, o nome de um pacote que a IA costuma inventar. **p. 32**

## GLOSSÁRIO RÁPIDO

CONTINUAÇÃO

**Smoke test.** Teste rápido de "sobe e responde?" logo após cada publicação. **p. 29**

**Software 1.0, 2.0 e 3.0.** Karpathy: 1.0 é código escrito à mão; 2.0, pesos aprendidos de dados; 3.0, prompts em linguagem natural como "programa". **p. 5**

**Spec Kit.** Toolkit de código aberto do GitHub para desenvolvimento orientado a especificação. **p. 23**

**Spike.** Protótipo descartável para aprender ou testar uma ideia; depois, reescreva com qualidade. **p. 11**

**SQL injection.** Entrada do usuário que altera uma consulta SQL; evitada com consultas parametrizadas. **p. 32**

**SSRF.** *Server-side request forgery*: o servidor é induzido a acessar endereços internos. **p. 42**

**Stack.** Conjunto de tecnologias de um projeto: linguagem, framework, banco e hospedagem.

**Subagente.** Agente auxiliar com contexto próprio: faz uma subtarefa e devolve só o resumo. **p. 20**

**SWE-bench.** Benchmark que mede se agentes resolvem issues reais de projetos de código aberto. **p. 8**

**System prompt.** Instruções de base que definem o comportamento do modelo ou do produto antes da conversa.

### T

**TDD.** *Test-driven development*: escrever o teste antes (falha), implementar o mínimo (passa) e refatorar. **p. 23**

**Temperatura.** Aleatoriedade na escolha do próximo token; valores baixos dão respostas mais previsíveis. **p. 8**

**Teste unitário.** Verifica uma função ou regra isolada; rápido e barato, forma a base da pirâmide. **p. 29**

**Tipos estáticos.** Anotações de tipo checadas antes de rodar (TypeScript, mypy): pegam uma classe inteira de erros de graça. **p. 29**

**Token.** Pedaço de texto (palavra, parte de palavra, símbolo) que o modelo processa; unidade de contexto e de cobrança. **p. 8**

**Tool poisoning.** Envenenamento de ferramentas: descrições de um servidor MCP malicioso trazem instruções ocultas ao agente. **p. 32**

### V

**Variável de ambiente.** Configuração ou segredo mantido fora do código, no ambiente ou em um `.env` (que vai no `.gitignore`). **p. 32**

**Vetor (banco vetorial).** Banco otimizado para guardar e buscar embeddings por similaridade; base de muitos sistemas RAG.

**Vibe coding.** Termo de Andrej Karpathy (2 fev 2025): "se entregar às vibes" e aceitar o código sem ler. Na versão ampla, programar com a IA escrevendo a maior parte. **p. 5**

**Vibe engineering.** Termo de Simon Willison (out 2025) para o uso profissional e disciplinado de agentes de código. **p. 5**

**Vulnerabilidade.** Falha explorável que compromete a segurança ou os dados de um sistema. **p. 32**

### W

**Walking skeleton.** Esqueleto mínimo que roda de ponta a ponta; o primeiro passo antes de acrescentar comportamentos. **p. 11**

**Webhook.** Chamada HTTP que um serviço faz a outro quando algo acontece (ex.: pagamento confirmado).

**Worktree.** *Git worktree*: várias pastas de trabalho do mesmo repositório, cada uma em sua branch — permite agentes em paralelo sem colisão. **p. 20**

### X

**XSS.** *Cross-site scripting*: injeção de script malicioso em páginas vistas por outras pessoas. **p. 32**

### Y

**YAGNI.** *You aren't gonna need it*: não construa o que ninguém pediu. A IA tende ao excesso de abstração. **p. 29**

### Z

**Zero-shot.** Pedir ao modelo sem dar exemplos (veja *few-shot*). **p. 14**

REFERÊNCIAS

# — FONTES E LEITURAS

Onde conferir os dados, ler os textos originais e continuar estudando.

Os dados e as afirmações deste guia se apoiam nas fontes abaixo. Estudos e ferramentas mudam rápido: ao citar um número, **abra a fonte** e confira a data, o método e as ressalvas.

SE VOCÊ SÓ PUDER LER CINCO

**Leituras essenciais**

1. **Simon Willison — "Not all AI-assisted programming is vibe coding"**: a distinção que organiza o debate.
2. **Claude Code — Best practices**: hábitos de contexto, verificação e planejamento que valem para qualquer agente.
3. **Anthropic — Effective context engineering for AI agents**: como pensar no que a IA vê a cada passo.
4. **METR — estudo de 2025 e atualização de 2026**: por que medir é melhor do que achar.
5. **Simon Willison — The lethal trifecta**: o modelo mental mais útil para a segurança de agentes.

## ORIGEM DO TERMO E DEFINIÇÕES

| FONTE | USADA EM | LINK |
|---|---|---|
| **Andrej Karpathy** · post original no X (2 fev 2025) | p. 5 | x.com/karpathy/status/1886192184808149383 |
| **Simon Willison** · "Not all AI-assisted programming is vibe coding (but vibe coding rocks)" (19 mar 2025) | p. 5 | simonwillison.net/2025/Mar/19/vibe-coding |
| **Simon Willison** · "Vibe engineering" (7 out 2025) | p. 5 | simonwillison.net/2025/Oct/7/vibe-engineering |
| **Wikipedia** · Vibe coding | p. 5 | en.wikipedia.org/wiki/Vibe_coding |
| **Merriam-Webster** · vibe coding | p. 5 | merriam-webster.com/dictionary/vibe coding |
| **CNN Business** · "Vibe coding named Collins Dictionary's word of the year" (6 nov 2025) | p. 5 | kvia.com/news/…/2025/11/06/vibe-coding-named-collins-dictionarys-word-of-the-year |
| **TechCrunch** · um quarto das startups do lote W25 da Y Combinator (6 mar 2025) | p. 5 · p. 36 | techcrunch.com/2025/03/06/a-quarter-of-startups-in-ycs-current-cohort-have-codebases-that-are-almost-entirely-ai-generated |
| **Business Insider** · "Andrew Ng says vibe coding is a bad name for a very real and exhausting job" (jun 2025) | p. 36 | businessinsider.nl/andrew-ng-says-vibe-coding-is-a-bad-name-for-a-very-real-and-exhausting-job |
| **The New Stack** · Karpathy propõe "agentic engineering" no lugar de "vibe coding" (2026) | p. 5 · p. 36 | thenewstack.io/vibe-coding-is-passe-karpathy-has-a-new-name-for-the-future-of-software |

## — FONTES E LEITURAS

CONTINUAÇÃO

### PESQUISAS E DADOS

| FONTE | USADA EM | LINK |
|---|---|---|
| **METR** · "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity" (10 jul 2025) | p. 36 | metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study |
| **METR** · "We are Changing our Developer Productivity Experiment Design" (24 fev 2026) | p. 36 | metr.org/blog/2026-02-24-uplift-update |
| **Stack Overflow** · Developer Survey 2025, seção de IA | p. 36 | survey.stackoverflow.co/2025/ai |
| **DORA** · State of AI-assisted Software Development 2025 | p. 36 | dora.dev/research/2025/dora-report |
| **Veracode** · 2025 GenAI Code Security Report (30 jul 2025) | p. 32 | veracode.com/blog/genai-code-security-report |
| **CodeRabbit** · State of AI vs Human Code Generation (17 dez 2025) | p. 36 | coderabbit.ai/blog/state-of-ai-vs-human-code-generation-report |
| **GitClear** · AI Copilot Code Quality: 2025 Data | p. 36 | gitclear.com/ai_assistant_code_quality_2025_research |
| **Anthropic** · "How AI assistance impacts the formation of coding skills" (29 jan 2026) | p. 36 | anthropic.com/research/AI-assistance-coding-skills |
| **arXiv 2406.10279** · "We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs" | p. 32 | arxiv.org/abs/2406.10279 |
| **Chroma** · "Context Rot: How Increasing Input Tokens Impacts LLM Performance" (2025) | p. 17 | research.trychroma.com/context-rot |

### TÉCNICAS, PROTOCOLOS E FERRAMENTAS

| FONTE | USADA EM | LINK |
|---|---|---|
| **Anthropic** · "Effective context engineering for AI agents" (29 set 2025) | p. 17 | anthropic.com/engineering/effective-context-engineering-for-ai-agents |
| **Claude Code** · Best practices | p. 11 · p. 17 · p. 20 | code.claude.com/docs/en/best-practices |
| **Anthropic** · guia de engenharia de prompt | p. 14 | docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview |
| **AGENTS.md** · formato aberto de instruções para agentes | p. 17 | agents.md |
| **Linux Foundation** · criação da Agentic AI Foundation | p. 20 | linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-agentic-ai-foundation |
| **Model Context Protocol** · site oficial | p. 20 | modelcontextprotocol.io |
| **GitHub Spec Kit** · toolkit de código aberto | p. 23 | github.com/github/spec-kit |
| **Kiro** · IDE agêntica da AWS | p. 23 | kiro.dev |

## — FONTES E LEITURAS

CONTINUAÇÃO

### SEGURANÇA, INCIDENTES E LEGISLAÇÃO

| FONTE | USADA EM | LINK |
|---|---|---|
| **OWASP** · Top 10 for LLM Applications 2025 | p. 32 | genai.owasp.org/llm-top-10 |
| **Simon Willison** · "The lethal trifecta for AI agents" (16 jun 2025) | p. 32 | simonwillison.net/2025/Jun/16/the-lethal-trifecta |
| **Pillar Security** · "Rules File Backdoor" (mar 2025) | p. 32 | pillar.security/blog/new-vulnerability-in-github-copilot-and-cursor-how-hackers-can-weaponize-code-agents |
| **NVD** · CVE-2025-48757 (Lovable) | p. 32 | nvd.nist.gov/vuln/detail/CVE-2025-48757 |
| **Fortune** · agente da Replit apaga banco de produção (23 jul 2025) | p. 32 | fortune.com/2025/07/23/ai-coding-tool-replit-wiped-database-called-it-a-catastrophic-failure |
| **Planalto** · Lei nº 13.709/2018 (LGPD) | p. 32 | planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm |

### PARA CONTINUAR ESTUDANDO

SE O MÓDULO 03 PARECEU CORRIDO

**Base de Git**

O livro **Pro Git** é gratuito e tem versão em português: git-scm.com/book/pt-br/v2.

SE O MÓDULO 02 OU 04 PARECEU CORRIDO

**Base de IA e prompts**

Releia os guias *Conceitos de IA* e *Git e GitHub* e use o glossário deste guia (**p. 45**) como revisão rápida.

**Sobre as fontes:** os links e os números refletem as fontes até **outubro de 2026** e podem ter sido atualizados depois. Onde a fonte ressalva a própria evidência (como a METR em 2026), mantivemos a ressalva no texto. Se um link mudar, busque pelo título.

# BONS ESTUDOS!




ESTUDAR · PESQUISAR · COMPARTILHAR · EVOLUIR

Vibe coding é uma ótima porta de entrada — mas quem entende o que a IA escreveu, testa e revisa é quem constrói coisas que duram. Continue estudando, praticando e compartilhando.